

Experiment [1]: [Linux OS Environment Setup]

Name: Paras bhall, Roll No.: 590029319, Date: 2025-10-30

AIM:

- To install and configure different Linux operating system environments on a Windows machine. We will use two distinct technologies: **Windows Subsystem for Linux (WSL)** for a lightweight command-line environment and **Oracle VirtualBox** for a full graphical virtual machine.

Requirements:

- A Windows 10/11 PC.
- Administrator access and **hardware virtualization enabled in the BIOS/UEFI**.
- An internet connection.

Theory:

- This experiment is designed to provide hands-on experience with two primary methods of running Linux on a Windows host. This is ideal for developers and system administrators who require a Linux command-line without the overhead of a full virtual machine.
- **Oracle VirtualBox**, on the other hand, is a traditional Type 2 hypervisor. It creates a complete, virtualized computer system on which a guest operating system (like Ubuntu or Linux Mint) can be installed. This method provides a fully isolated environment, complete with a graphical user interface (GUI).

Procedure & Observations

Part 1: Installing and Configuring WSL (Ubuntu)

Exercise 1: [Installing WSL on Windows]

- **Task Statement:** Enable the required Windows features and install the Ubuntu Linux distribution using a single command.
- **Explanation:** This demonstrates how the modern `wsl --install` command simplifies the entire setup process, automating what previously required multiple manual steps.
- **Command(s):**

```
wsl --install -d ubuntu
```

- **Observation:** The command automatically enabled the "Virtual Machine Platform" and "Windows Subsystem for Linux" optional components. It then proceeded to download the Ubuntu distribution. The system requested a reboot to complete the final stage of the installation.

Exercise 2: [Configuring the Ubuntu Distribution]

- **Task Statement:** After the initial installation and reboot, configure the Ubuntu environment by creating a new user account.
- **Explanation:** This step is crucial for security and user management within the Linux environment. The new UNIX username and password created are separate from the Windows user account.
- **Observation:** Upon reboot, a terminal window opened automatically. It prompted for a "New UNIX username" and a password. After entering the credentials, the setup was complete and the command-line interface became available.

Exercise 3: [Verifying WSL Installation]

- **Task Statement:** Confirm that the WSL installation is successful and the Ubuntu distribution is ready for use.
- **Explanation:** This command provides a simple way to list all installed WSL distributions, showing their names, versions, and current state.
- **Command(s):**

```
wsl -l -v
```

- **Output:**

NAME	STATE	VERSION
* Ubuntu	Running	2

- **Observation:** The output confirmed that Ubuntu was correctly installed and was currently in a "Running" state, with version 2 (indicating it is running on the WSL 2 architecture).

Part 2: Installing VirtualBox and a Linux VM (Linux Mint)

Exercise 4: [Installing Oracle VirtualBox]

- **Task Statement:** Download and install the Oracle VirtualBox hypervisor on the Windows host machine.
- **Procedure:** The VirtualBox installer was downloaded from the official website. Permission to install device software for network interfaces was granted.
- **Observation:** The VirtualBox application was successfully installed on the Windows system, along with the necessary drivers to support virtual machines.

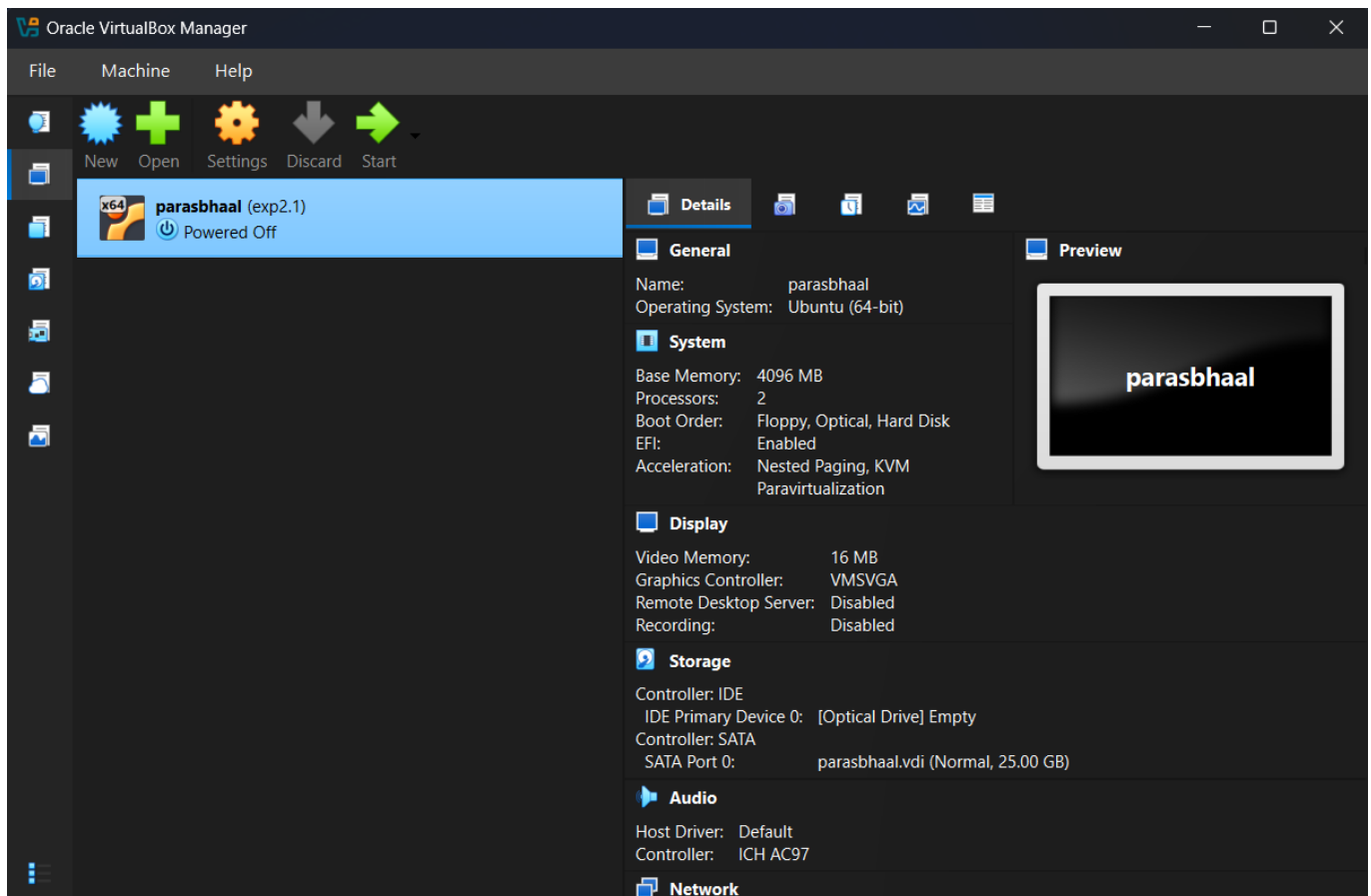
Exercise 5: [Creating a Virtual Machine]

- **Task Statement:** Create a new virtual machine to host the Linux Mint operating system.
- **Procedure:** In the VirtualBox Manager, a new VM was created. The name was set to "Linux Mint", and a downloaded **.iso** file was selected as the installation medium. Hardware resources were configured with **4096 MB RAM** and **2 CPUs**. A new dynamically allocated virtual hard disk of **25 GB** was created.

- **Observation:** The VM was configured with the specified resources, creating a virtualized hardware environment ready to receive an operating system.

Exercise 6: [Installing Linux Mint]

- **Task Statement:** Install the Linux Mint OS on the virtual machine.
- **Procedure:** The newly created VM was started, which booted directly from the Linux Mint **.iso**.
- **Observation:** The installation proceeded without issues, partitioning the virtual disk and copying the OS files. The process was identical to a standard installation on a physical computer.



Result

- The experiment was successfully completed by setting up two distinct Linux environments. **Windows Subsystem for Linux** and a complete **virtual machine** with Linux Mint. This project demonstrated proficiency in using both a compatibility layer and a full hypervisor to meet different virtualization needs.

Experiment [2]: [Linux file systems permissions and essential commands]

Name: PARAS BHALL Roll.: 590029319 Date: 2025-30-10

AIM:

- [To Learn linux file systems permissions and essential commands]

Requirements:

- [Any Linux Distro, any kind of text editor (vs code, vim, notepad, nano, etc,)]

Theory:

- [Basic Linux file systems permissions and essential commands]

Procedure & Observations

TASK 1: [Directory Navigation]

Task Statement:

- [Create a directory called test_project in your home directory, then create subdirectories docs, scripts, and data inside it. Navigate to the scripts directory and display your current path.]

Explanation:

- [Use mkdir to create the wanted directory we can use cd to navigate and use pwd to show current path]

Command(s):

```
"" mkdir test_project cd test_project mkdir docs scripts data cd scripts pwd ""
```

Output:

```
parasjaat34@parasbhaal: ~$ mkdir test_project
parasjaat34@parasbhaal: ~$ cd test_project
parasjaat34@parasbhaal: ~/test_project$ mkdir docs scripts data
parasjaat34@parasbhaal: ~/test_project$ cd scripts
parasjaat34@parasbhaal: ~/test_project/scripts$ pwd
/home/parasjaat34/test_project/scripts
parasjaat34@parasbhaal: ~/test_project/scripts$
```

TASK 2: [File Creation and Content]

Task Statement:

- [Create three files in the docs directory: readme.txt, notes.txt, and todo.txt. Add the text "Project documentation" to readme.txt and "Important notes" to notes.txt. Display the contents of both files.]

Explanation:

- [We can use touch to create empty files and using echo "text" > file.txt to add content to a file and using cat to display file contents]

Command(s):

```
cd docs
touch readme.txt notes.txt todo.txt
echo "Project documentation" > readme.txt
echo "Important notes" > notes.txt
cat notes.txt
cat readme.txt
```

Output:

```
parasjaat34@parasbhaal: ~/$ mkdir docs
parasjaat34@parasbhaal:~$ cd docs
parasjaat34@parasbhaal:~/docs$ echo "Project documentation" > readme.txt
parasjaat34@parasbhaal:~/docs$ echo "Important notes" > notes.txt
parasjaat34@parasbhaal:~/docs$ cat notes.txt
Important notes
parasjaat34@parasbhaal:~/docs$ cat readme.txt
Project documentation
parasjaat34@parasbhaal:~/docs$
```

TASK 3: [File Operations]

Task Statement:

- [Copy readme.txt to the data directory and rename the copy to project_info.txt. Then move todo.txt from docs to scripts directory.]

Explanation:

- [- We can use the cp source destination to copy files and using the mv oldname newname to rename files also using the same command mv file directory/ to move files to another directory we can also combine copy and rename: cp file.txt newdir/newname.txt]

Command(s):

```
cp readme.txt data/project_info.txt
```

Output:

```
parasjaat34@parasbhaal: ~/docs
parasjaat34@parasbhaal:~$ mkdir docs
parasjaat34@parasbhaal:~$ cd docs
parasjaat34@parasbhaal:~/docs$ echo "Project documentation" > readme.txt
parasjaat34@parasbhaal:~/docs$ echo "Important notes" > notes.txt
parasjaat34@parasbhaal:~/docs$ cat notes.txt
Important notes
parasjaat34@parasbhaal:~/docs$ cat readme.txt
Project documentation
parasjaat34@parasbhaal:~/docs$
```

TASK 4: [File Permissions]

Task Statement:

- [Create a shell script file called backup.sh in the scripts directory. Add the content `#!/bin/bash` and echo "Backup complete" to it. Make the file executable only for the owner.]

Explanation:

- [Using `chmod u+x filename` we can make the file executable for user only using `ls -l` to check for permissions also script files typically need executable permission to run]

Command(s):

```
cd scripts
touch backup.sh > echo "Backup complete"
chmod u+x backup.sh
```

Output:

```
parasjaat34@parasbhaal: ~/scripts
parasjaat34@parasbhaal:~$ mkdir scripts
parasjaat34@parasbhaal:~$ cd scripts
parasjaat34@parasbhaal:~/scripts$ touch backup.sh > echo "Backup complete"
parasjaat34@parasbhaal:~/scripts$ chmod u+x backup.sh
parasjaat34@parasbhaal:~/scripts$ ls -l
total 0
-rw-rw-r-- 1 parasjaat34 parasjaat34 0 Oct 30 18:15 'Backup complete'
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 backup.sh
-rw-rw-r-- 1 parasjaat34 parasjaat34 0 Oct 30 18:15 echo
parasjaat34@parasbhaal:~/scripts$ chmod 777 backup.sh
parasjaat34@parasbhaal:~/scripts$ ls -l
total 0
-rw-rw-r-- 1 parasjaat34 parasjaat34 0 Oct 30 18:15 'Backup complete'
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 backup.sh
-rw-rw-r-- 1 parasjaat34 parasjaat34 0 Oct 30 18:15 echo
parasjaat34@parasbhaal:~/scripts$ chmod 777 *
parasjaat34@parasbhaal:~/scripts$ ls -l
total 0
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 'Backup complete'
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 backup.sh
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 echo
parasjaat34@parasbhaal:~/scripts$
```

```
parasjaat34@parasbhaal: ~/scripts
parasjaat34@parasbhaal:~$ mkdir scripts
parasjaat34@parasbhaal:~$ cd scripts
parasjaat34@parasbhaal:~/scripts$ touch backup.sh > echo "Backup complete"
parasjaat34@parasbhaal:~/scripts$ chmod u+x backup.sh
parasjaat34@parasbhaal:~/scripts$ ls -l
total 0
-rw-rw-r-- 1 parasjaat34 parasjaat34 0 Oct 30 18:15 'Backup complete'
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 backup.sh
-rw-rw-r-- 1 parasjaat34 parasjaat34 0 Oct 30 18:15 echo
parasjaat34@parasbhaal:~/scripts$ chmod 777 backup.sh
parasjaat34@parasbhaal:~/scripts$ ls -l
total 0
-rw-rw-r-- 1 parasjaat34 parasjaat34 0 Oct 30 18:15 'Backup complete'
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 backup.sh
-rw-rw-r-- 1 parasjaat34 parasjaat34 0 Oct 30 18:15 echo
parasjaat34@parasbhaal:~/scripts$ chmod 777 *
parasjaat34@parasbhaal:~/scripts$ ls -l
total 0
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 'Backup complete'
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 backup.sh
-rwxrwxrwx 1 parasjaat34 parasjaat34 0 Oct 30 18:15 echo
parasjaat34@parasbhaal:~/scripts$
```

TASK 5: [File Viewing]

Task Statement:

- [Create a file called numbers.txt with numbers 1 to 20 (each on a new line). Display only the first 5 lines, then only the last 3 lines, then search for lines containing the number "1".]

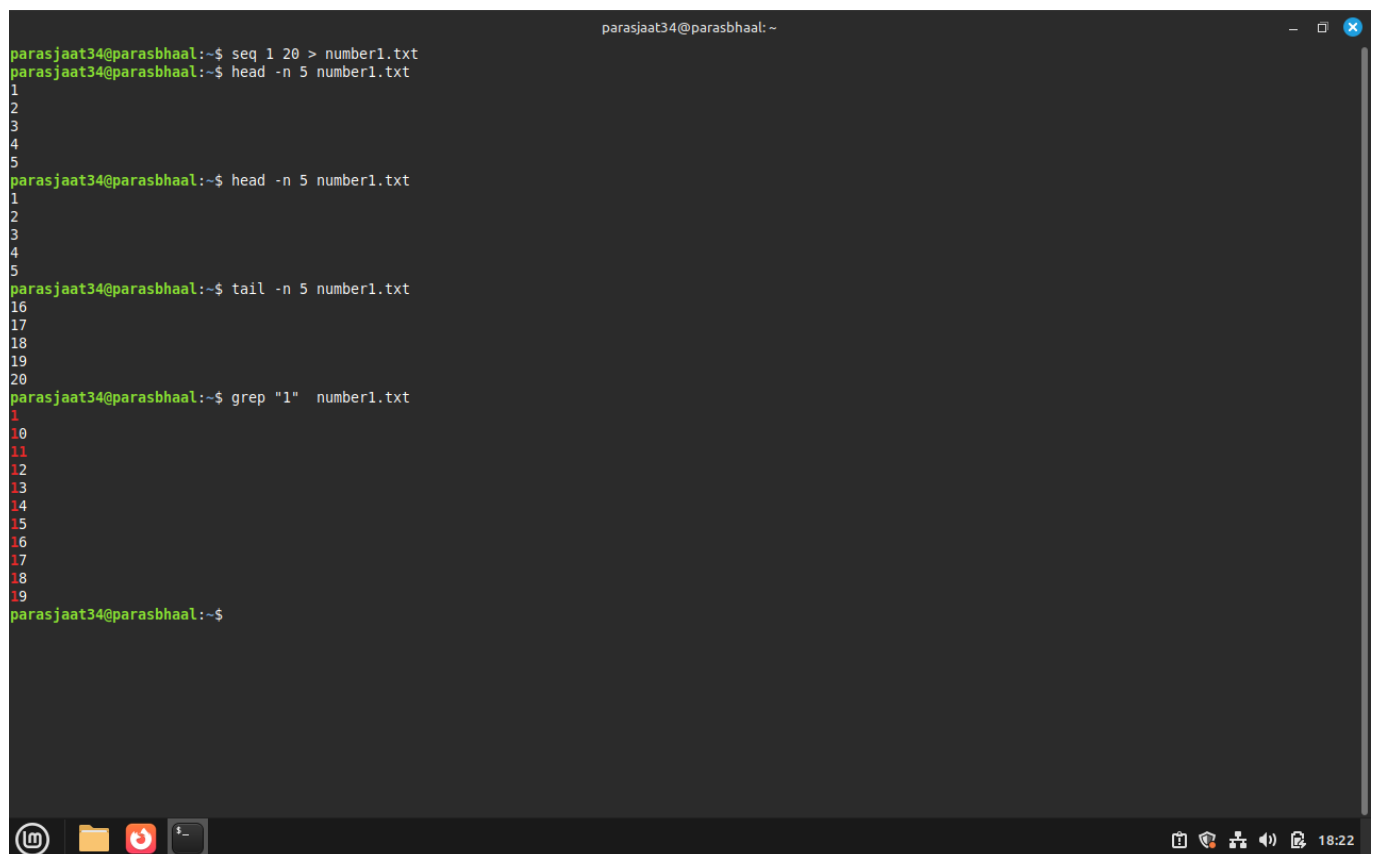
Explanation:

- [I can quickly generate a list of numbers by running `seq 1 20 > numbers.txt`. To check the first few numbers, I use `head -n 5` to see the first 5 lines, and `tail -n 3` to see the last 3 lines. If I want to find all numbers containing a "1", I can use `grep "1"`. Alternatively, I could create the list manually by using multiple `echo` commands.]

Command(s):

```
seq 1 20 > numbers.txt
head -n 5
tail -n 3
grep "1"
```

Output:



```
parasjaat34@parasbhaal:~$ seq 1 20 > number1.txt
parasjaat34@parasbhaal:~$ head -n 5 number1.txt
1
2
3
4
5
parasjaat34@parasbhaal:~$ head -n 5 number1.txt
1
2
3
4
5
parasjaat34@parasbhaal:~$ tail -n 5 number1.txt
16
17
18
19
20
parasjaat34@parasbhaal:~$ grep "1" number1.txt
1
10
11
12
13
14
15
16
17
18
19
parasjaat34@parasbhaal:~$
```

TASK 6: [Text Editing]

Task Statement:

- [Using nano, create a file called `config.txt` with the following content:

Database=localhost Port=5432 Username=admin

Save the file and then display its contents.]

Explanation:

- [I open a file in Nano using nano filename.txt and type my content normally. Once I'm done, I press Ctrl+O to save the file and Ctrl+X to exit Nano. After that, I use cat to check the contents and make sure everything was saved correctly.]

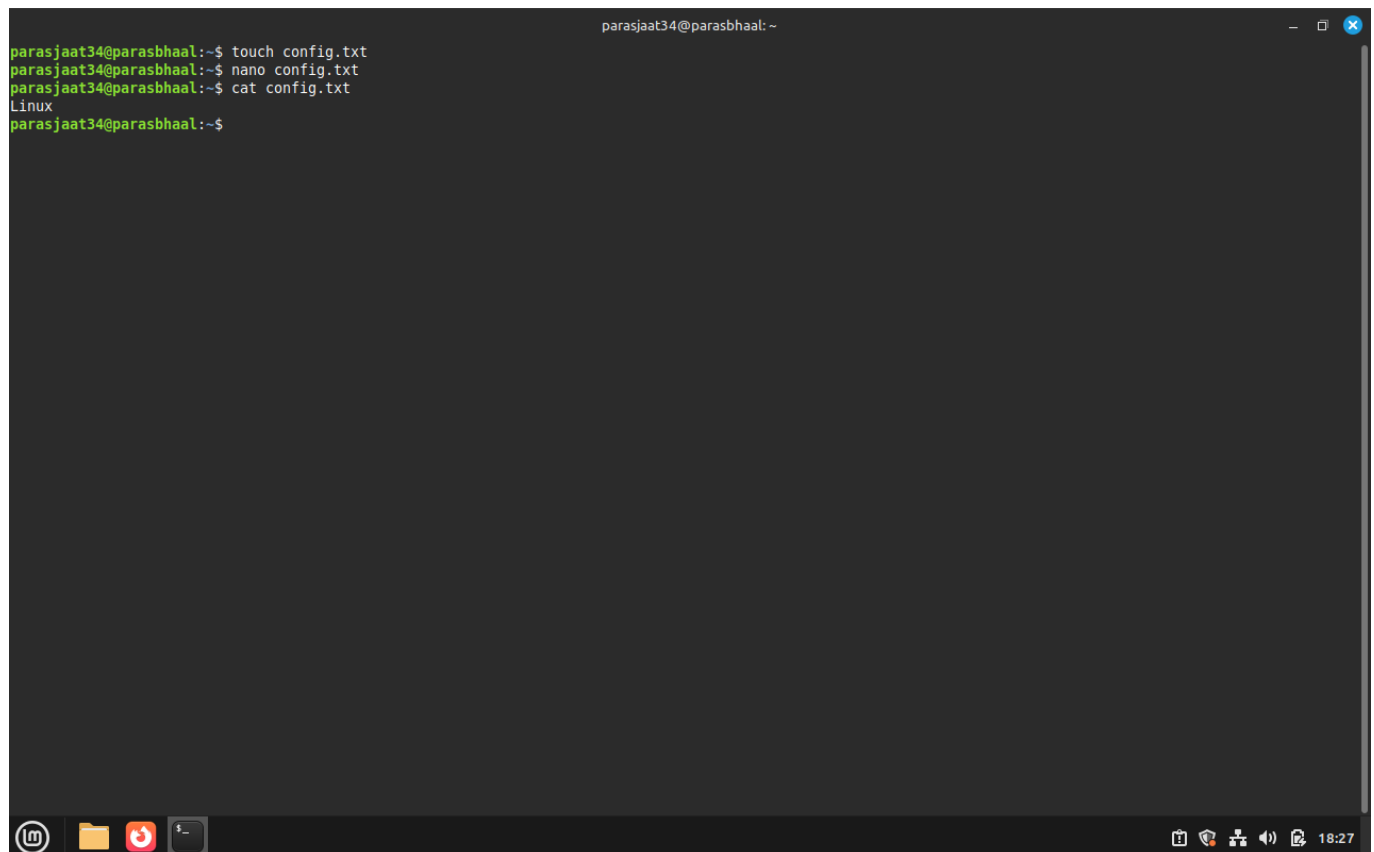
Command(s):

```
vim config.txt  
cat config.txt
```

Alternatively

```
nano config.txt  
cat config.txt
```

Output:



```
parasjaat34@parasbhaal: ~  
parasjaat34@parasbhaal:~$ touch config.txt  
parasjaat34@parasbhaal:~$ nano config.txt  
parasjaat34@parasbhaal:~$ cat config.txt  
Linux  
parasjaat34@parasbhaal:~$
```

The screenshot shows a terminal window with a dark background. The user 'parasjaat34' is at the 'parasbhaal' machine. The commands executed are: 'touch config.txt', 'nano config.txt', and 'cat config.txt'. The output of 'cat config.txt' is 'Linux'. The terminal window has a title bar with standard Linux window controls and a system tray at the bottom showing various icons and the time '18:27'.

TASK 7: [System Information]

Task Statement:

- [Create a file called system_info.txt that contains: your username, current date, your current directory, and disk usage information in human-readable format.]

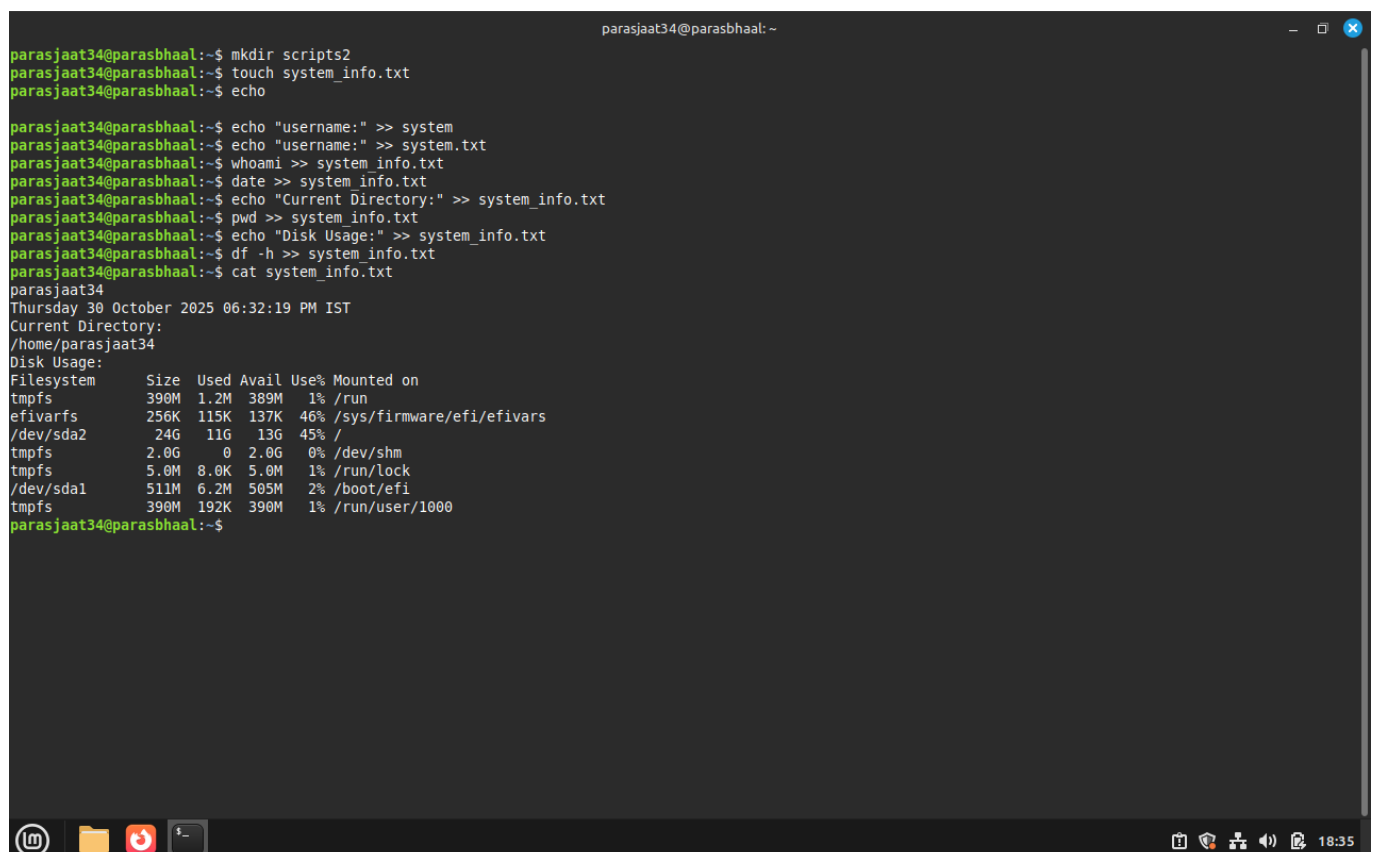
Explanation:

- [I can use whoami to check my username, date to see the current date, and pwd to know my current directory. To check disk usage, I use df -h. I can save the output of any command to a file by using redirection like command >> filename.txt. If I want to add labels, I use echo like this: echo "Username:" >> file.txt.]

Command(s):

```
cd scripts
touch system_info.txt
echo "Username:" >> system_info.txt
whoami >> system_info.txt
echo "Date:" >> system_info.txt
date >> system_info.txt
echo "Current Directory:" >> system_info.txt
pwd >> system_info.txt
echo "Disk Usage:" >> system_info.txt
df -h >> system_info.txt
```

Output:



```
parasjaat34@parasbhaal:~$ mkdir scripts2
parasjaat34@parasbhaal:~$ touch system_info.txt
parasjaat34@parasbhaal:~$ echo
parasjaat34@parasbhaal:~$ echo "username:" >> system
parasjaat34@parasbhaal:~$ echo "username:" >> system.txt
parasjaat34@parasbhaal:~$ whoami >> system_info.txt
parasjaat34@parasbhaal:~$ date >> system_info.txt
parasjaat34@parasbhaal:~$ echo "Current Directory:" >> system_info.txt
parasjaat34@parasbhaal:~$ pwd >> system_info.txt
parasjaat34@parasbhaal:~$ echo "Disk Usage:" >> system_info.txt
parasjaat34@parasbhaal:~$ df -h >> system_info.txt
parasjaat34@parasbhaal:~$ cat system_info.txt
parasjaat34
Thursday 30 October 2025 06:32:19 PM IST
Current Directory:
/home/parasjaat34
Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           390M  1.2M  389M   1% /run
efivarfs        256K  115K  137K  46% /sys/firmware/efi/efivars
/dev/sda2        24G   11G   13G  45% /
tmpfs           2.0G   0  2.0G   0% /dev/shm
tmpfs           5.0M   8.0K  5.0M   1% /run/lock
/dev/sda1       511M   6.2M  505M   2% /boot/efi
tmpfs           390M  192K  390M   1% /run/user/1000
parasjaat34@parasbhaal:~$
```

TASK 8: [File Organisation]

Task Statement:

- [In your test_project directory, create a backup folder. Copy all .txt files from all subdirectories into this backup folder. Then list all files in the backup folder with detailed information.]

Explanation:

- [I can use `find . -name "*.txt"` to locate all .txt files. Alternatively, I can navigate to each directory and copy files manually. To copy multiple files at once, I use `cp file1.txt file2.txt destination/`. If I want detailed information about the files, I use `ls -la`. The wildcard `*.txt` helps me match all files that end with .txt.]

Command(s):

```
cp test_project/data/project_info.txt    test_project/docs/notes.txt
test_project/docs/readme.txt            test_project/docs/todo.txt
test_project/scripts/config.txt          test_project/scripts/numbers.txt
test_project/scripts/system_info.txt     test_project/scripts/todo.txt    backup/
```

Output:

```

parasjaat34@parasbhaal: ~
parasjaat34@parasbhaal:~$ mkdir readme.txt
parasjaat34@parasbhaal:~$ mkdir todo.txt
parasjaat34@parasbhaal:~$ cp -r readme.txt todo.txt scripts/
parasjaat34@parasbhaal:~$ ls -la
total 252
drwxr-x--- 22 parasjaat34 parasjaat34 4096 Oct 30 18:38 .
drwxr-xr-x  3 root         root         4096 Aug 26 22:04 ..
-rw-rw-r--  1 parasjaat34 parasjaat34   237 Sep  9 17:27 '\ '
-rw-rw-r--  1 parasjaat34 parasjaat34    10 Aug 29 16:21 abc.txt
-rwxrwxr-x  1 parasjaat34 parasjaat34 16104 Sep 20 11:27 a.out
-rw-r--r--  1 parasjaat34 parasjaat34   434 Oct 30 18:02 .bash_history
-rw-r--r--  1 parasjaat34 parasjaat34   220 Aug 26 22:04 .bash_logout
-rw-r--r--  1 parasjaat34 parasjaat34  3771 Aug 26 22:04 .bashrc
drwx----- 11 parasjaat34 parasjaat34  4096 Oct 30 17:58 .cache
drwxr-xr-x 17 parasjaat34 parasjaat34  4096 Oct 30 17:58 .config
-rw-rw-r--  1 parasjaat34 parasjaat34    7 Oct 30 18:27 config.txt
drwxr-xr-x  2 parasjaat34 parasjaat34  4096 Sep 20 11:26 Desktop
-rw-r--r--  1 parasjaat34 parasjaat34   27 Aug 26 22:11 .dmrc
drwxrwxr-x  2 parasjaat34 parasjaat34  4096 Oct 30 18:04 docs
drwxr-xr-x  2 parasjaat34 parasjaat34  4096 Aug 26 22:11 Documents
drwxr-xr-x  2 parasjaat34 parasjaat34  4096 Oct 30 17:54 Downloads
-rw-rw-r--  1 parasjaat34 parasjaat34   426 Sep 20 11:27 exp3.c
-rw-r--r--  1 parasjaat34 parasjaat34   22 Aug 26 22:04 .gtkr-2.0
-rw-r--r--  1 parasjaat34 parasjaat34   516 Aug 26 22:04 .gtkr-xfce
-rw-r--r--  1 parasjaat34 parasjaat34   20 Aug 29 17:21 .lessht
drwxrwxr-x  3 parasjaat34 parasjaat34  4096 Sep  5 16:07 .linuxmint
drwxr-xr-x  4 parasjaat34 parasjaat34  4096 Aug 26 22:11 .local
-rw-rw-r--  1 parasjaat34 parasjaat34   218 Sep 14 01:29 luv.c
-rw-rw-r--  1 parasjaat34 parasjaat34   20 Aug 29 17:08 me.txt
drwx----- 4 parasjaat34 parasjaat34  4096 Sep 20 11:34 .mozilla
drwxr-xr-x  2 parasjaat34 parasjaat34  4096 Aug 26 22:11 Music
-rw-rw-r--  1 parasjaat34 parasjaat34   51 Oct 30 18:21 number1.txt
drwxr-xr-x  2 parasjaat34 parasjaat34  4096 Sep 20 11:34 Pictures
-rw-r--r--  1 parasjaat34 parasjaat34   807 Aug 26 22:04 .profile
drwxr-xr-x  2 parasjaat34 parasjaat34  4096 Aug 26 22:11 Public
-rw-rw-r--  1 parasjaat34 parasjaat34   215 Sep 14 01:21 qwe.c
drwxrwxr-x  2 parasjaat34 parasjaat34  4096 Oct 30 18:38 readme.txt
-rwxrwxr-x  1 parasjaat34 parasjaat34    60 Sep  9 16:57 script1.sh
-rwxrwxr-x  1 parasjaat34 parasjaat34   232 Sep  9 17:32 script2.sh
-rw-r--r--  1 parasjaat34 parasjaat34 12288 Sep  9 17:39 .script2.sh.swp
-rw-rw-r--  1 parasjaat34 parasjaat34    49 Sep  9 16:35 script2.txt
drwxrwxr-x  4 parasjaat34 parasjaat34  4096 Oct 30 18:39 scripts
drwxrwxr-x  2 parasjaat34 parasjaat34  4096 Oct 30 18:38 scripts2

```

TASK 9: [Process and History]

Task Statement:

- [Display your command history and count how many commands you've executed. Then show the top 10 most recent commands.]

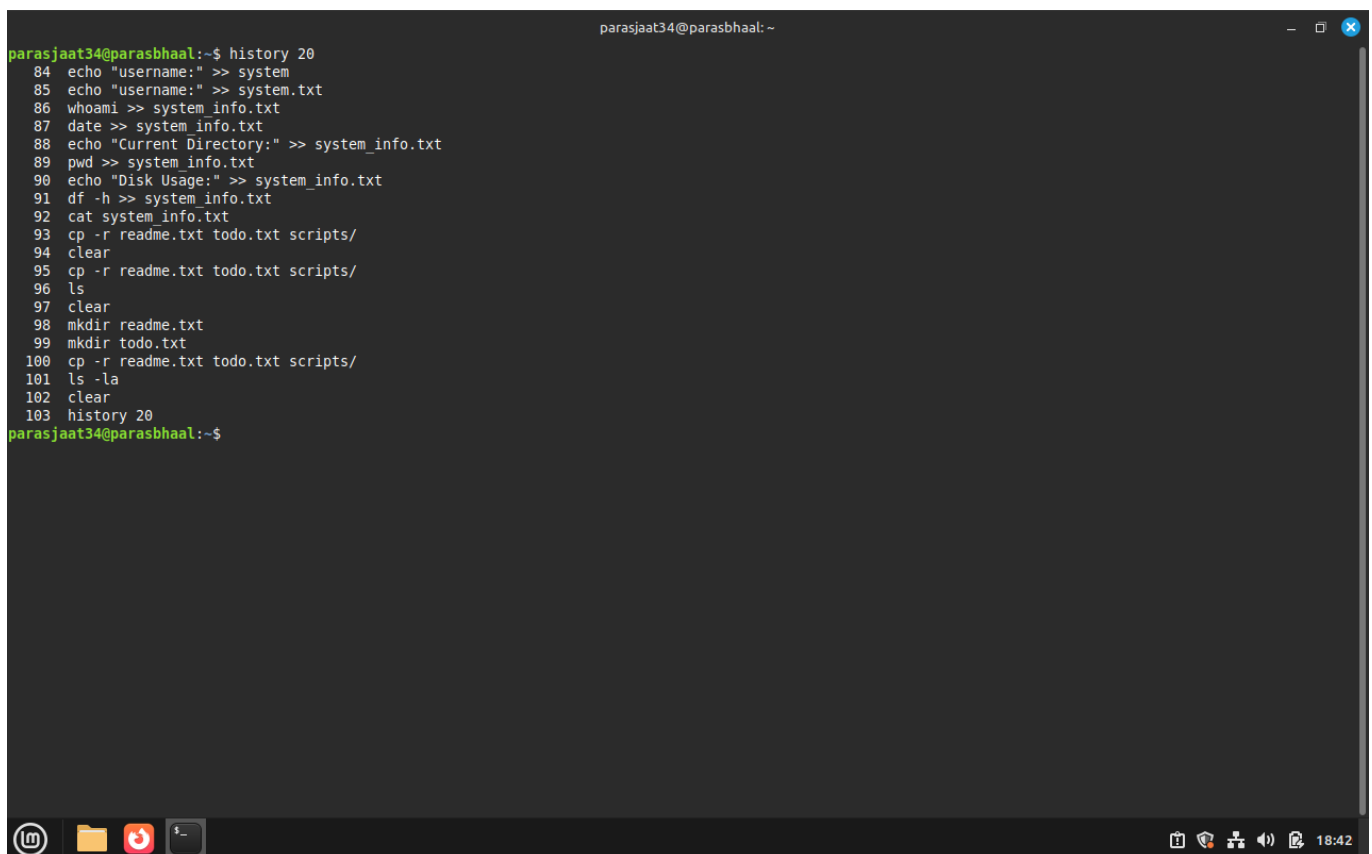
Explanation:

- [I can use history to see all the commands I've typed. To count the total number of commands, I use history | wc -l. If I want to view just the last 10 commands, I can use history 10 or history | tail -10. The wc -l command simply counts the number of lines in the output.]

Command(s):

```
history 10
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' showing the output of the 'history 20' command. The output lists 20 commands with their corresponding line numbers (84 to 103). The commands include 'echo', 'whoami', 'date', 'pwd', 'df', 'cat', 'cp', 'clear', 'ls', 'mkdir', and 'history'. The terminal has a dark background with green text for the prompt and command, and white text for the output. The window includes standard Linux terminal window controls (minimize, maximize, close) and a taskbar at the bottom with icons for a terminal, file manager, and other applications, along with a system clock showing 18:42.

```
parasjaat34@parasbhaal:~$ history 20
84 echo "username:" >> system
85 echo "username:" >> system.txt
86 whoami >> system_info.txt
87 date >> system_info.txt
88 echo "Current Directory:" >> system_info.txt
89 pwd >> system_info.txt
90 echo "Disk Usage:" >> system_info.txt
91 df -h >> system_info.txt
92 cat system_info.txt
93 cp -r readme.txt todo.txt scripts/
94 clear
95 cp -r readme.txt todo.txt scripts/
96 ls
97 clear
98 mkdir readme.txt
99 mkdir todo.txt
100 cp -r readme.txt todo.txt scripts/
101 ls -la
102 clear
103 history 20
parasjaat34@parasbhaal:~$
```

TASK 10: [Comprehensive Cleanup]

Task Statement:

- [Set the permissions of your backup.sh script to be readable, writable, and executable by owner, readable and executable by group, and readable by others. Then create a summary file that lists the total number of files and directories in your entire test_project.]

Explanation:

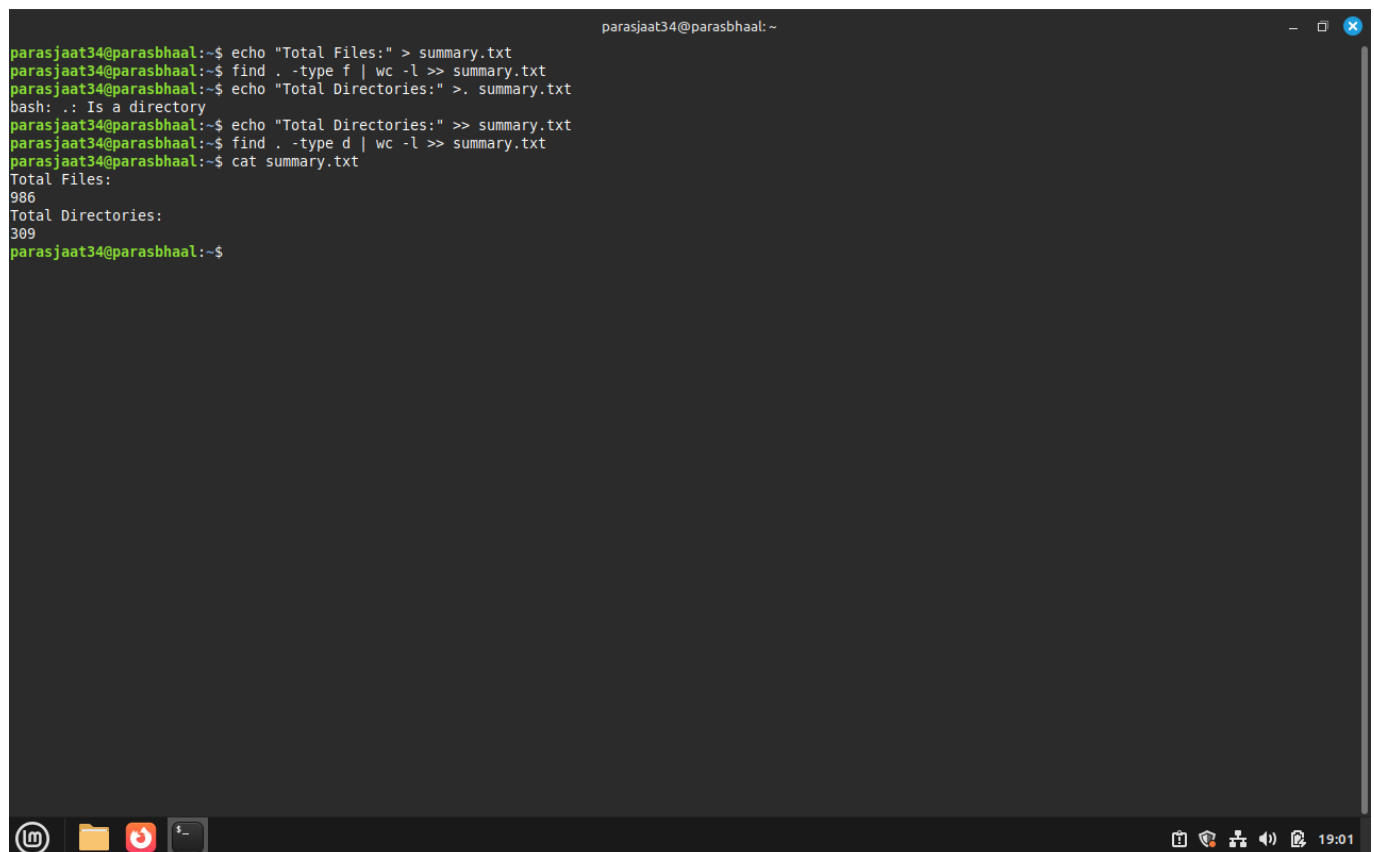
- [I can set permissions for backup.sh using chmod 754 backup.sh to give rwxr-xr-- permissions. Alternatively, I can use chmod u=rwx,g=rx,o=r backup.sh. To count all files, I use find . -type f | wc -l, and to count directories, I use find . -type d | wc -l. If I want to see the full directory structure recursively, I use ls -R. I can also combine multiple commands with && or save the outputs to a summary file for later reference.]

Command(s):

```
chmod 754 backup.sh

echo "Total files:" > summary.txt
find . -type f | wc -l >> summary.txt
echo "Total directories:" >> summary.txt
find . -type d | wc -l >> summary.txt
```

Output:



```
parasjaat34@parasbhaal:~$ echo "Total Files:" > summary.txt
parasjaat34@parasbhaal:~$ find . -type f | wc -l >> summary.txt
parasjaat34@parasbhaal:~$ echo "Total Directories:" >> summary.txt
bash: .: Is a directory
parasjaat34@parasbhaal:~$ echo "Total Directories:" >> summary.txt
parasjaat34@parasbhaal:~$ find . -type d | wc -l >> summary.txt
parasjaat34@parasbhaal:~$ cat summary.txt
Total Files:
986
Total Directories:
309
parasjaat34@parasbhaal:~$
```

Experiment 3: Linux File Manipulation and System Manipulation I

Name: Paras Bhall Roll No.: 590029319 Date: 2025-10-30

Aim:

- To practice Linux file manipulation commands like `touch`, `cp`, `mv`, `rm`, `cat`, `less`, `head`, `tail`.
- To explore file permissions and ownership with `ls -l`, `chmod`, `chown`, and `chgrp`.
- To search and filter files using `find` and `grep`.
- To understand archiving and compression with `tar`, `gzip`, and `gunzip`.
- To create and manage links (`ln`) for both hard and symbolic links.

Requirements

- A Linux machine with bash shell (Ubuntu/Fedora/other).
- User privileges to create, modify, and delete files and directories.
- Access to system utilities like `tar`, `gzip`, `grep`, and `find`.

Theory

Linux file management involves creating, copying, moving, removing, and viewing files. File permissions and ownership ensure secure access control. Searching and filtering tools like `grep` and `find` help locate information efficiently. Archiving with `tar` and compression with `gzip` reduce storage usage and simplify file transfer. Links (`ln`) allow multiple references to the same file data (hard links) or path references (symbolic links).

Procedure & Observations

Exercise 1: Creating and Managing Files

Task Statement:

Create files and manage timestamps using `touch`.

Command(s):

```
touch newfile.txt
touch file1.txt file2.txt file3.txt
touch -t 202401151430 dated_file.txt
```

Output:



Exercise 2: Copying, Moving, and Deleting Files

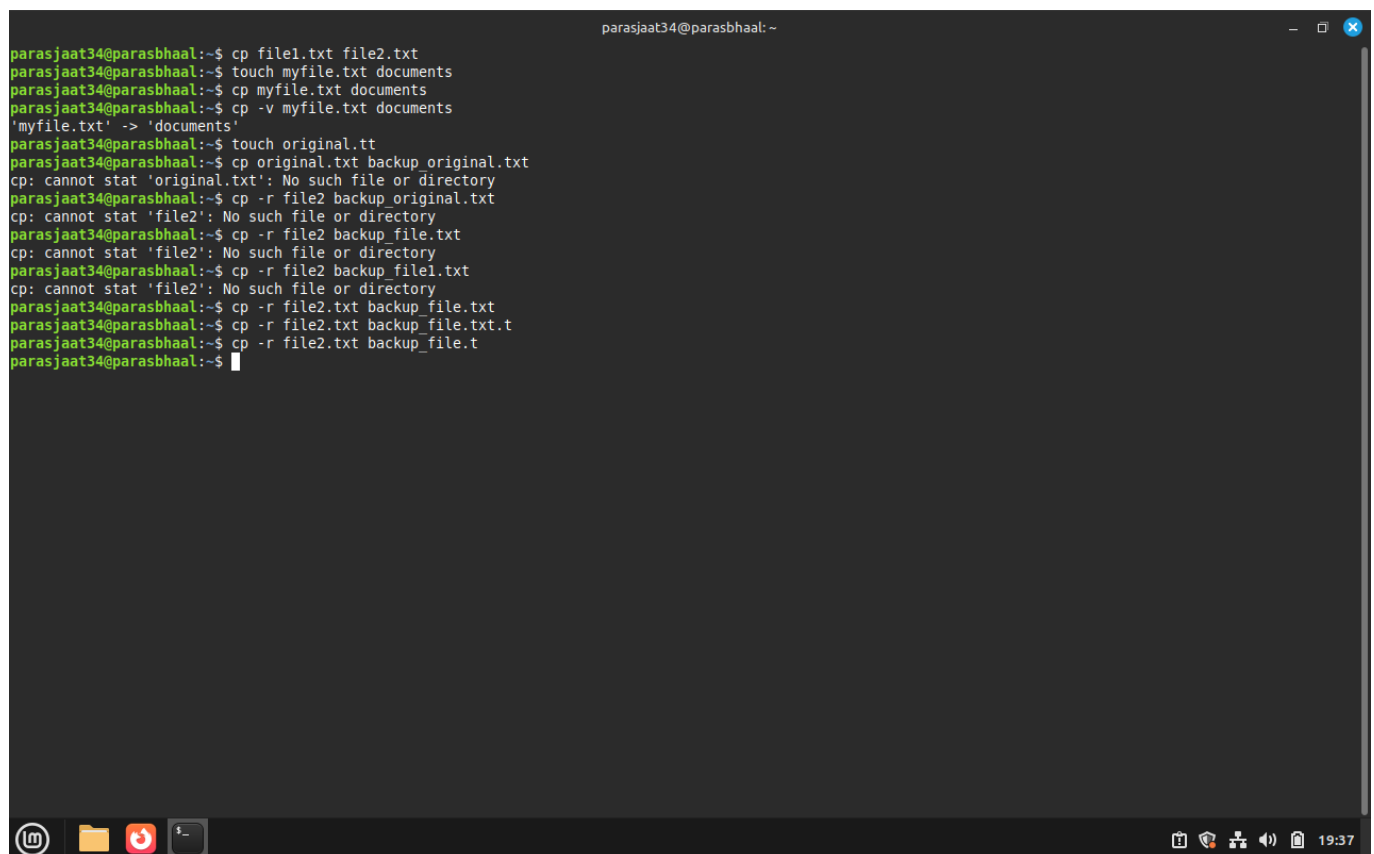
Task Statement:

Use **cp**, **mv**, and **rm** to copy, rename, move, and delete files and directories.

Command(s):

```
cp document.txt backup_document.txt
mv oldname.txt newname.txt
rm unwanted_file.txt
rm -r old_directory/
```

Output:



```
parasjaat34@parasbhaal:~$ cp file1.txt file2.txt
parasjaat34@parasbhaal:~$ touch myfile.txt documents
parasjaat34@parasbhaal:~$ cp myfile.txt documents
parasjaat34@parasbhaal:~$ cp -v myfile.txt documents
'myfile.txt' -> 'documents'
parasjaat34@parasbhaal:~$ touch original.tt
parasjaat34@parasbhaal:~$ cp original.txt backup_original.txt
cp: cannot stat 'original.txt': No such file or directory
parasjaat34@parasbhaal:~$ cp -r file2 backup_original.txt
cp: cannot stat 'file2': No such file or directory
parasjaat34@parasbhaal:~$ cp -r file2 backup_file.txt
cp: cannot stat 'file2': No such file or directory
parasjaat34@parasbhaal:~$ cp -r file2 backup_file1.txt
cp: cannot stat 'file2': No such file or directory
parasjaat34@parasbhaal:~$ cp -r file2.txt backup_file.txt
parasjaat34@parasbhaal:~$ cp -r file2.txt backup_file.txt.t
parasjaat34@parasbhaal:~$ cp -r file2.txt backup_file.t
parasjaat34@parasbhaal:~$
```

Exercise 3: Viewing File Contents

Task Statement:

Display file contents using **cat**, **less**, **head**, and **tail**.

Command(s):

```
cat filename.txt
less /var/log/syslog
head -n 5 filename.txt
tail -n 20 filename.txt
tail -f /var/log/syslog
```


Output:

```
parasjaat34@parasbhaal: ~  
parasjaat34@parasbhaal:~$ rm myfile.txt  
parasjaat34@parasbhaal:~$ ls  
'\'  
abc.txt      backup_file.txt      Desktop  Downloads  file3.txt  newfile.txt  Public  script1.sh  scripts2  system.txt  test_project  
a.out        backup_file.txt.t    docs     exp3.c     luv.c     number1.txt  qwe.c   script2.sh  summary.txt  Templates  todo.txt  
backup_file.t  config.txt           documents file1.txt  me.txt    original.tt  readme.txt script2.txt system      test.c     Videos  
backup_file.t  dated_file.txt      Documents file2.txt  Music     Pictures     script   script2.txt system_info.txt testdir  
parasjaat34@parasbhaal:~$
```

Exercise 4: File Permissions and Ownership

Task Statement:

Explore file permissions and ownership with `ls -l`, `chmod`, `chown`, and `chgrp`.

Command(s):

```
ls -l  
chmod 755 script.sh  
chmod u+x script.sh  
sudo chown newuser:newgroup file.txt  
chgrp developers project.txt
```

Output:

```
parasjaat34@parasbhaal: ~  
parasjaat34@parasbhaal:~$ mkdir dir1  
parasjaat34@parasbhaal:~$ touch dir1/file3.txt  
parasjaat34@parasbhaal:~$ ls dir1  
file3.txt  
parasjaat34@parasbhaal:~$ mkdir file4  
parasjaat34@parasbhaal:~$ mkdir dir2  
parasjaat34@parasbhaal:~$ ls dir2  
parasjaat34@parasbhaal:~$ touch file.txt  
parasjaat34@parasbhaal:~$ mkdir file1  
parasjaat34@parasbhaal:~$ mv file1 touch.txt  
parasjaat34@parasbhaal:~$ ls  
'\'  
abc.txt      backup_file.txt.t  dir2      exp3.c      file.txt     number1.txt  readme.txt  scripts      system.txt  todo.txt  
a.out        config.txt         docs      file1.txt   luv.c        original.tt  script      scripts2     Templates  touch.txt  
backup_file.t dated_file.txt     Documents file2.txt   me.txt       Pictures     script1.sh  summary.txt  test.c      Videos  
backup_file.txt Desktop            Documents file3.txt   Music        Public       script2.sh  system      testdir     test_project  
backup_file.txt dir1              Downloads file4       newfile.txt  qwe.c       script2.txt  system_info.txt  
parasjaat34@parasbhaal:~$
```

Exercise 5: File Searching with `find`

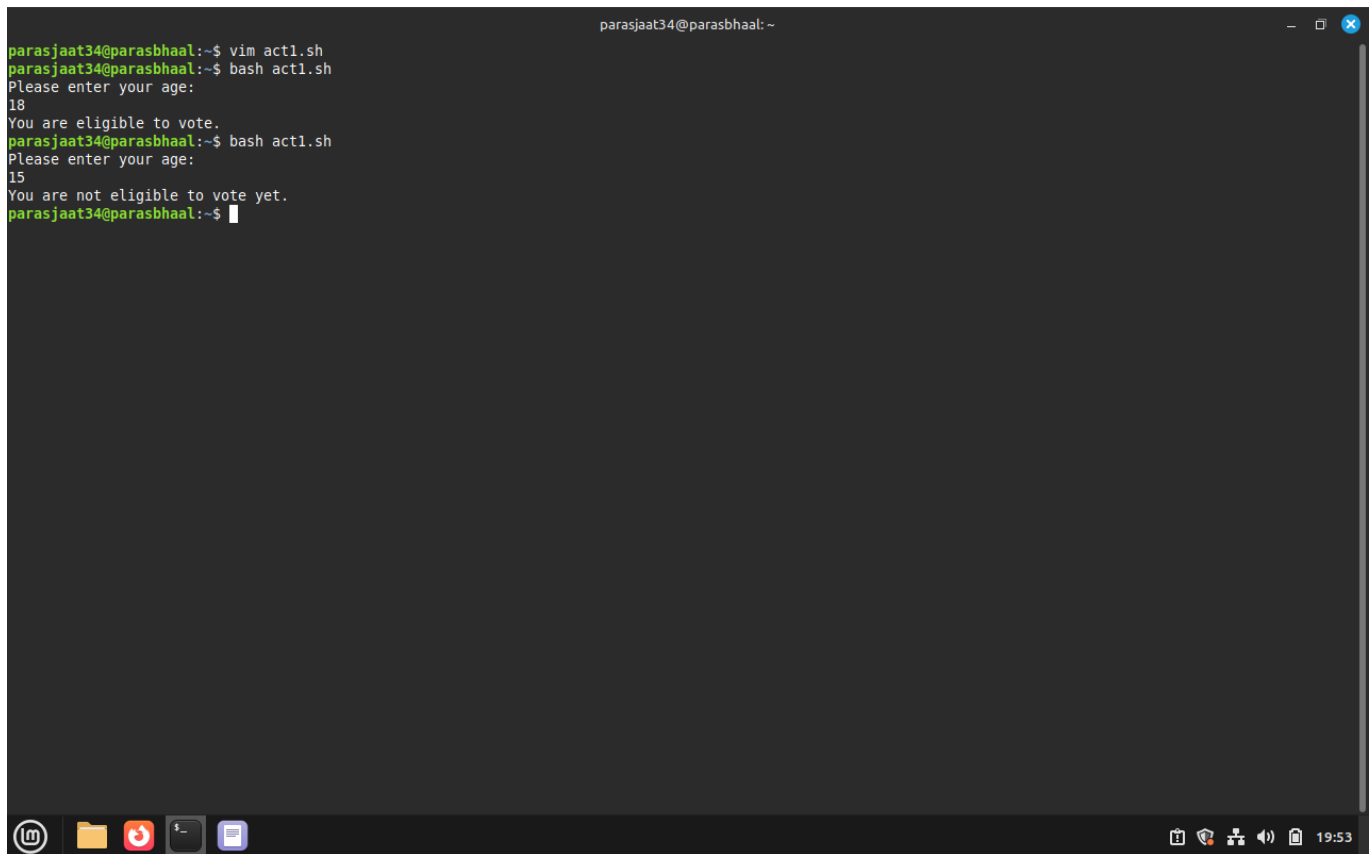
Task Statement:

Search files by name, type, size, and permissions using `find`.

Command(s):

```
find /home -name "*.txt"  
find /home -type f -size +100M  
find /etc -name "*conf*"  
find /tmp -type f -empty -delete
```

Output:



```
parasjaat34@parasbhaal:~$ vim act1.sh
parasjaat34@parasbhaal:~$ bash act1.sh
Please enter your age:
18
You are eligible to vote.
parasjaat34@parasbhaal:~$ bash act1.sh
Please enter your age:
15
You are not eligible to vote yet.
parasjaat34@parasbhaal:~$
```

Exercise 6: Pattern Searching with **grep**

Task Statement:

Search for patterns in files using **grep**.

Command(s):

```
grep "error" /var/log/syslog
grep -i "Error" logfile.txt
grep -r "function" ~/code/
grep -n "TODO" *.txt
```

Output:

```
parasjaat34@parasbhaal:~$ touch logfile.txt
parasjaat34@parasbhaal:~$ grep "error" /var/log/syslog
2025-08-26T22:11:06.356862+05:30 parasbhaal alsactl[674]: alsa-lib main.c:1554:(snd_use_case_mgr_open) error: failed to import hw:0 use case configuration -2
2025-08-26T22:11:06.365099+05:30 parasbhaal udisksd[639]: Error probing device: Error sending ATA command IDENTIFY PACKET DEVICE to '/dev/sr0': Unexpected sense data returned:#0120000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....#0120010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
.....#012 (g-io-error-quark, 0)
grep: /var/log/syslog: binary file matches
parasjaat34@parasbhaal:~$
```

Exercise 7: Archiving and Compression

Task Statement:

Create and extract archives using **tar**, compress and decompress with **gzip/gunzip**.

Command(s):

```
tar -czf backup.tar.gz /home/user/documents
tar -xzf backup.tar.gz -C /restore/
gzip largefile.txt
gunzip largefile.txt.gz
```

Output:



Exercise 8: Creating Links

Task Statement:

Create and test hard and symbolic links using **ln**.

Command(s):

```
echo "Hello" > original.txt
ln original.txt hardlink.txt
ln -s original.txt symlink.txt
ls -li original.txt hardlink.txt symlink.txt
```

Output:

```

parasjaat34@parasbhaal:~$ ln file.txt hardlink_file
parasjaat34@parasbhaal:~$ ls -li
total 160
1189298 -rw-rw-r-- 1 parasjaat34 parasjaat34  237 Sep  9 17:27 '\ '
1189178 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Aug 29 16:21 abc.txt
1180018 -rw-rw-r-- 1 parasjaat34 parasjaat34  147 Oct 30 19:53 act1.sh
1187631 -rwxrwxr-x 1 parasjaat34 parasjaat34 16104 Sep 20 11:27 a.out
1180330 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:35 backup_file.t
1180327 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:34 backup_file.txt
1180328 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:34 backup_file.txt.t
1179999 -rw-rw-r-- 1 parasjaat34 parasjaat34    7 Oct 30 18:27 config.txt
1180004 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Jan 15 2024 dated_file.txt
1188824 drwxr-xr-x 2 parasjaat34 parasjaat34  4096 Sep 20 11:26 Desktop
1180010 drwxrwxr-x 2 parasjaat34 parasjaat34  4096 Oct 30 19:45 dir1
1180811 drwxrwxr-x 2 parasjaat34 parasjaat34  4096 Oct 30 19:47 dir2
1190154 drwxrwxr-x 2 parasjaat34 parasjaat34  4096 Oct 30 18:04 docs
1180062 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:30 documents
1188837 drwxr-xr-x 2 parasjaat34 parasjaat34  4096 Aug 26 22:11 Documents
1188827 drwxr-xr-x 2 parasjaat34 parasjaat34  4096 Oct 30 17:54 Downloads
1189144 -rw-rw-r-- 1 parasjaat34 parasjaat34  426 Sep 20 11:27 exp3.c
1180015 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:03 file1.txt
1180016 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:29 file2.txt
1180017 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:40 file3.txt
1180808 drwxrwxr-x 2 parasjaat34 parasjaat34  4096 Oct 30 19:47 file4
1180812 -rw-rw-r-- 2 parasjaat34 parasjaat34    0 Oct 30 19:48 file.txt
1180812 -rw-rw-r-- 2 parasjaat34 parasjaat34    0 Oct 30 19:48 hardlink_file
1179716 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:54 logfile.txt
1189297 -rw-rw-r-- 1 parasjaat34 parasjaat34  218 Sep 14 01:29 luv.c
1189191 -rw-rw-r-- 1 parasjaat34 parasjaat34   20 Aug 29 17:08 me.txt
1188838 drwxr-xr-x 2 parasjaat34 parasjaat34  4096 Aug 26 22:11 Music
1180014 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:02 newfile.txt
1179940 -rw-rw-r-- 1 parasjaat34 parasjaat34   51 Oct 30 18:21 number1.txt
1180066 -rw-rw-r-- 1 parasjaat34 parasjaat34    0 Oct 30 19:30 original.tt
1188839 drwxr-xr-x 2 parasjaat34 parasjaat34  4096 Sep 20 11:34 Pictures
1188829 drwxr-xr-x 2 parasjaat34 parasjaat34  4096 Aug 26 22:11 Public
1189369 -rw-rw-r-- 1 parasjaat34 parasjaat34  215 Sep 14 01:21 qwe.c
1180006 drwxrwxr-x 2 parasjaat34 parasjaat34  4096 Oct 30 18:38 readme.txt
1179689 drwxrwxr-x 4 parasjaat34 parasjaat34  4096 Oct 30 18:54 script
1189280 -rwxrwxr-x 1 parasjaat34 parasjaat34    60 Sep  9 16:57 script1.sh
1189277 -rwxrwxr-x 1 parasjaat34 parasjaat34   232 Sep  9 17:32 script2.sh
1189279 -rw-rw-r-- 1 parasjaat34 parasjaat34    49 Sep  9 16:35 script2.txt
1179989 drwxrwxr-x 4 parasjaat34 parasjaat34  4096 Oct 30 18:39 scripts
1179938 drwxrwxr-x 2 parasjaat34 parasjaat34  4096 Oct 30 18:30 scripts2

```

Result

- Successfully created, copied, moved, and deleted files.
- Practiced viewing file contents and monitoring logs.
- Explored file permissions and ownership management.
- Used **find** and **grep** to locate and filter data.
- Created archives and compressed files.
- Demonstrated both hard and symbolic links.

Challenges Faced & Learning Outcomes

- Challenge 1: Accidentally deleted files with **rm** without **-i**. Learned to use **rm -i** for safety.
- Challenge 2: Remembering numeric vs symbolic permissions in **chmod**. Fixed through repeated practice.

Learning:

- Gained practical skills with file manipulation and permission commands.
- Learned how to efficiently search files and patterns in Linux.

- Understood how to archive and compress files for better storage management.
- Understood differences between hard and symbolic links.

Conclusion

This experiment provided hands-on experience with core Linux file management, permissions, searching, archiving, and linking. These are foundational skills for effective Linux system administration and daily usage.

Experiment [4]: [Bash Scripting]

Name:Paras bhall, Roll No.: 590029319, Date: 2025-10-30

AIM:

- [To Learn Basics of Bash Scripting.]

Requirements:

- [Any Linux Distro, any kind of text editor (vs code, vim, notepad, nano, etc)]

Theory:

- [Learning the basics of bash scripting.]

Procedure & Observations

Exercise 1: [Hello World Script]

Task Statement:

- [Basic Usage of Shell Scripts]

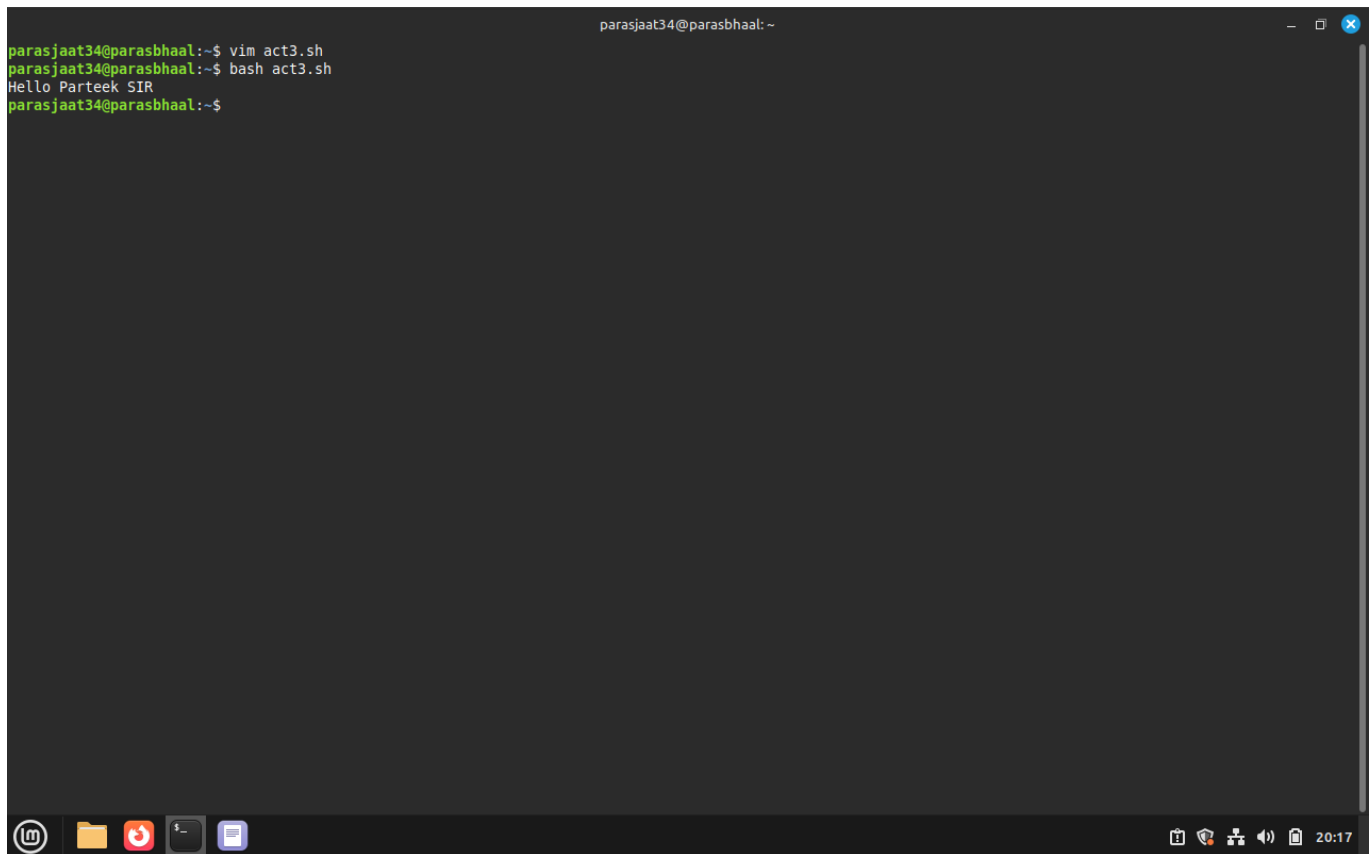
Explanation:

- [Writing Begginer level Shell Scripts]

Command(s):

```
#!/bin/bash  
echo "Hello, World!"
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' with standard window controls. The terminal shows a sequence of commands and their outputs: 'vim act3.sh' is executed, followed by 'bash act3.sh', which outputs 'Hello Parteek SIR'. The prompt returns to '~\$'. The terminal has a dark background with green text. At the bottom, there is a taskbar with icons for a terminal, files, a web browser, and other applications, along with system status icons on the right showing the time as 20:17.

```
parasjaat34@parasbhaal:~$ vim act3.sh
parasjaat34@parasbhaal:~$ bash act3.sh
Hello Parteek SIR
parasjaat34@parasbhaal:~$
```

Exercise 2: [Personalized Greeting Script]

Task Statement:

- [Basic Shell Script to callout user defined function.]

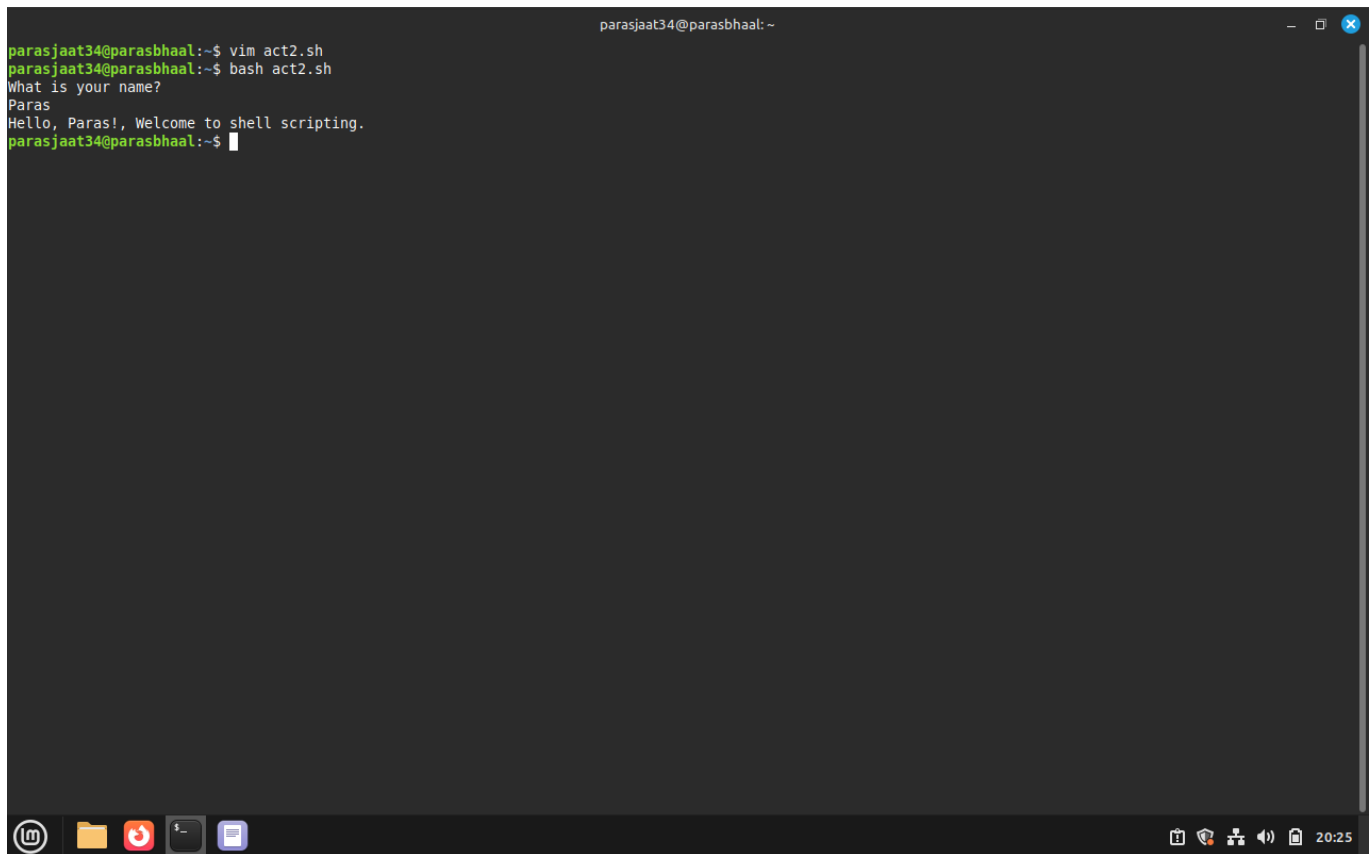
Explanation:

- [This Shell script will take input from user and store it in a variable and then call the variable which will output the stored value.]

Command(s):

```
#!/bin/bash
echo "What is your name?"
read name
echo "Hello, $name! Welcome to Shell Scripting."
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' showing the execution of a shell script. The user runs 'vim act2.sh' and then 'bash act2.sh'. The script prompts 'What is your name?' and the user enters 'Paras'. The script then outputs 'Hello, Paras!, Welcome to shell scripting.' and returns to the prompt.

```
parasjaat34@parasbhaal:~$ vim act2.sh
parasjaat34@parasbhaal:~$ bash act2.sh
What is your name?
Paras
Hello, Paras!, Welcome to shell scripting.
parasjaat34@parasbhaal:~$
```

Exercise 3: [Arithmetic Operations in Shell Scripting]

Task Statement:

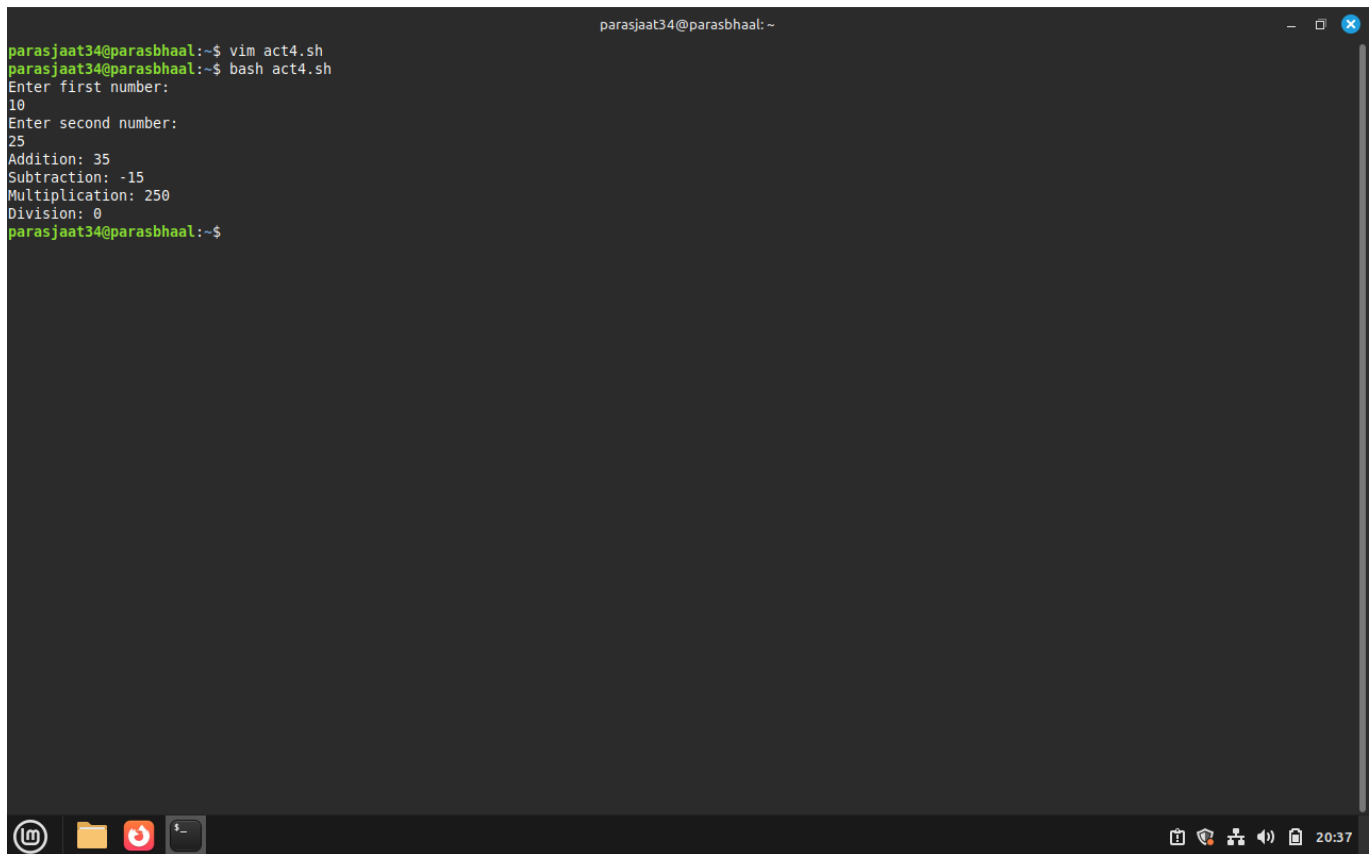
- [Using Basic Arithmetic Operations in Shell Scripts]

Command(s):

```
#!/bin/bash
echo "Enter first number: "
read num1
echo "Enter second number: "
read num2

echo "Addition: $((num1 + num2))"
echo "Subtraction: $((num1 - num2))"
echo "Multiplication: $((num1 * num2))"
echo "Division: $((num1 / num2))"
```

Output:

A screenshot of a terminal window with a dark background. The window title is 'parasjaat34@parasbhaal: ~'. The user has executed 'vim act4.sh' and then 'bash act4.sh'. The script prompts for two numbers: 'Enter first number:' (10) and 'Enter second number:' (25). It then displays the results of arithmetic operations: 'Addition: 35', 'Subtraction: -15', 'Multiplication: 250', and 'Division: 0'. The prompt returns to 'parasjaat34@parasbhaal:~\$'. The terminal has a standard Linux desktop environment at the bottom with icons for a terminal, folder, and other applications, and a system tray showing the time as 20:37.

```
parasjaat34@parasbhaal:~$ vim act4.sh
parasjaat34@parasbhaal:~$ bash act4.sh
Enter first number:
10
Enter second number:
25
Addition: 35
Subtraction: -15
Multiplication: 250
Division: 0
parasjaat34@parasbhaal:~$
```

Exercise 4: * [Voting Eligibility]

Task Statement:

- [Using Conditionals in Shell script]

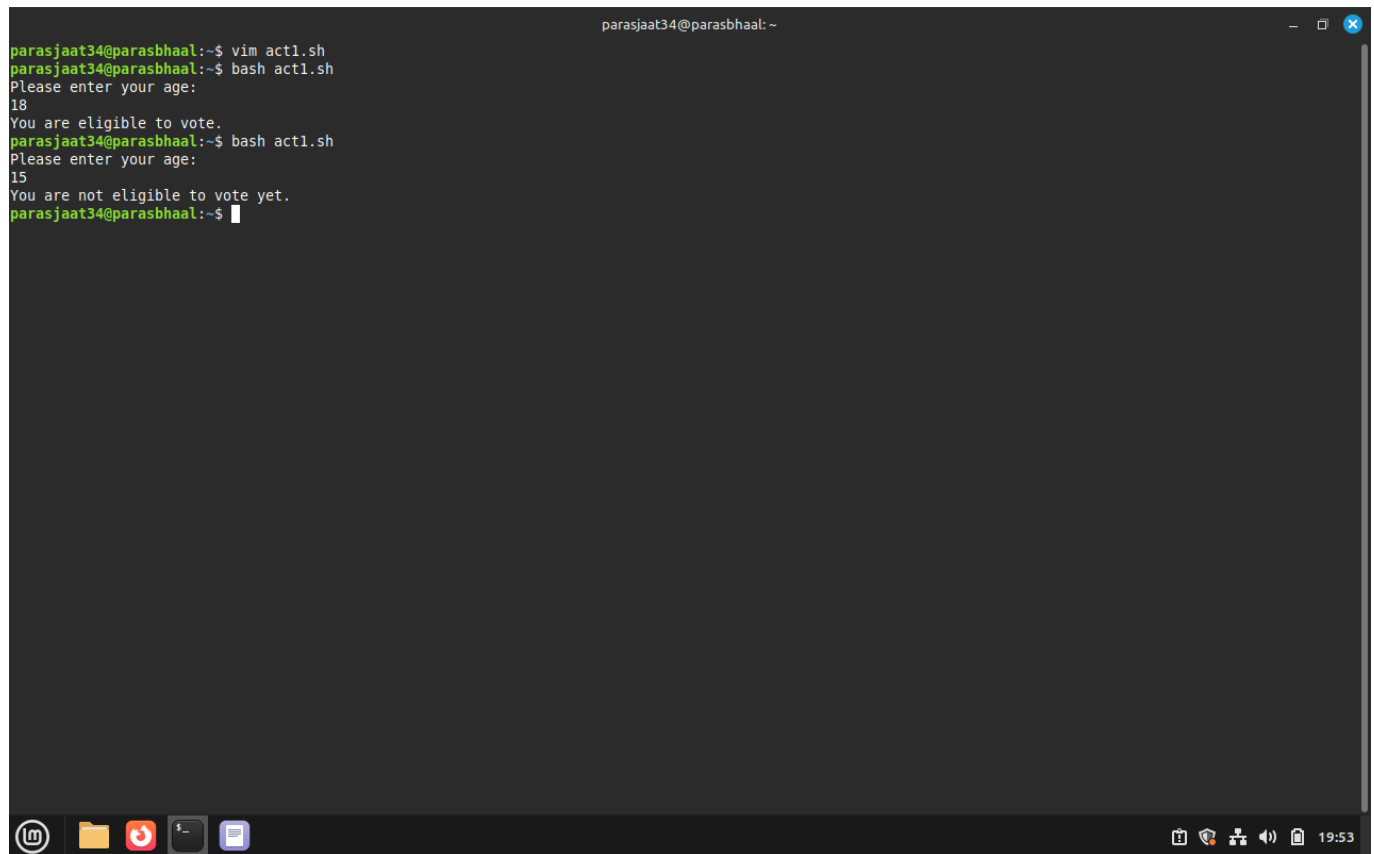
Command(s):

```
#!/bin/bash
echo "What is your age?"
read age
if [ $age -ge 18 ]; then

    echo "You are eligible to vote!"

else
    echo "You are not eligible to vote!"
fi
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' with standard window controls. The terminal shows a user editing 'act1.sh' with vim, then running it twice. The first run prompts for age, receives '18', and outputs 'You are eligible to vote.' The second run receives '15' and outputs 'You are not eligible to vote yet.' The terminal has a dark background and a taskbar at the bottom with icons for a terminal, folder, reddit, and other applications. The system clock shows 19:53.

```
parasjaat34@parasbhaal:~$ vim act1.sh
parasjaat34@parasbhaal:~$ bash act1.sh
Please enter your age:
18
You are eligible to vote.
parasjaat34@parasbhaal:~$ bash act1.sh
Please enter your age:
15
You are not eligible to vote yet.
parasjaat34@parasbhaal:~$
```

Result

- The Exercises were successfully completed for Basic Shell Scripting

Experiment [5]: [Shell Programming]

Name:Paras bhall Roll.590029319: Date: 2025-10-30

AIM:

- [To Learn Basic Conditional Statements in Bash Scripting]

Requirements:

- [Any Linux Distro, any kind of text editor (vs code, vim, notepad, nano, etc)]

Theory:

- [Basic usage of conditions and arrays in bash scripting.]

Procedure & Observations

Exercise 1: [Prime Number Check]

Task Statement:

- [To check if the number given by the user is a prime number or not.]

Explanation:

- [using if else loop wap to check if the number is a prime number or not.]

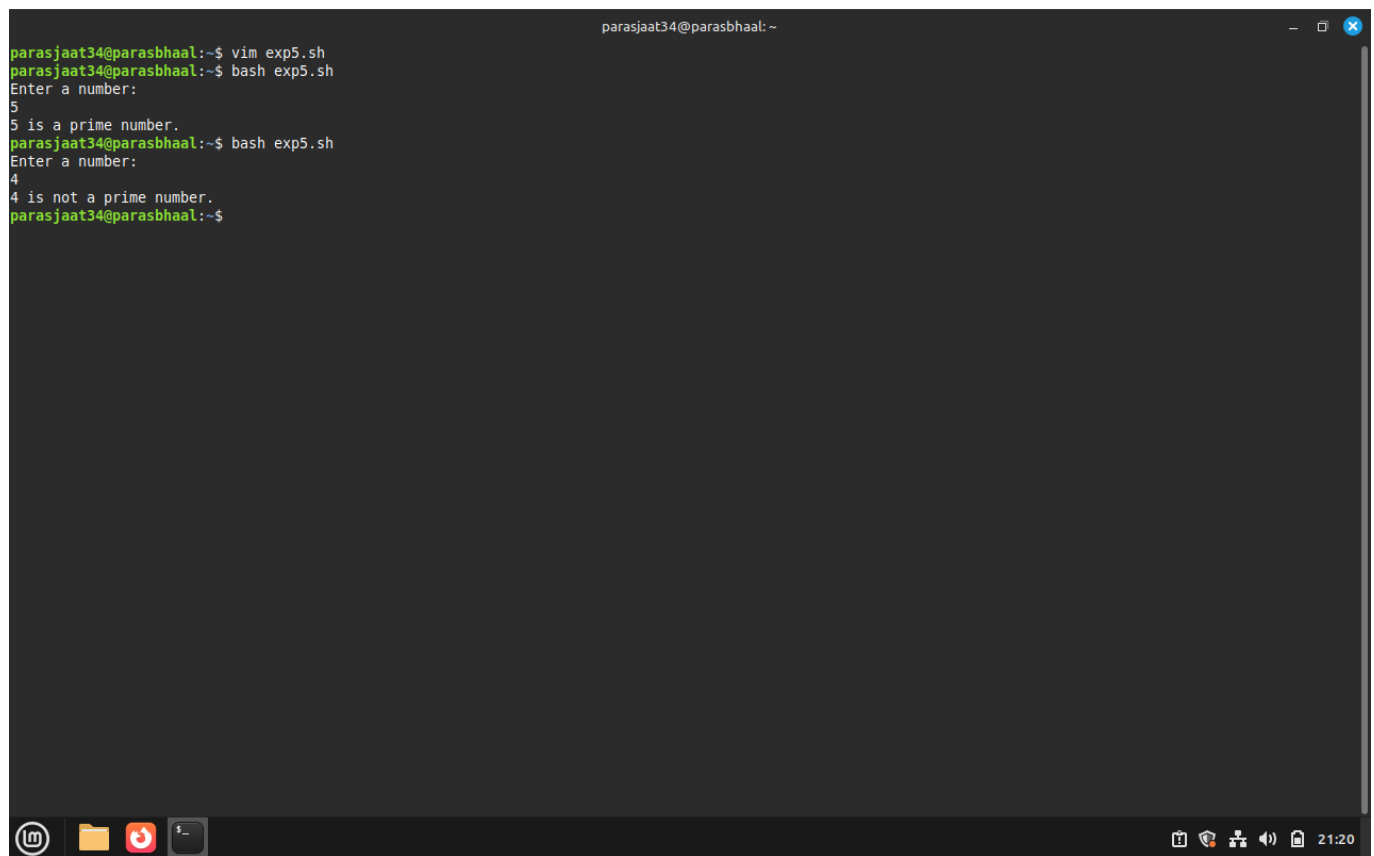
Command(s):

```
#!/bin/bash
echo "Enter a number: "
read num
flag=0

for ((i=2; i<=num/2; i++))
do
    if [ $((num % i)) -eq 0 ]
    then
        flag=1
        break
    fi
done

if [ $flag -eq 0 ]
then
    echo "$num is a prime number."
else
    echo "$num is not a prime number."
fi
```

Output:



```
parasjaat34@parasbhaal:~$ vim exp5.sh
parasjaat34@parasbhaal:~$ bash exp5.sh
Enter a number:
5
5 is a prime number.
parasjaat34@parasbhaal:~$ bash exp5.sh
Enter a number:
4
4 is not a prime number.
parasjaat34@parasbhaal:~$
```

Exercise 2: [Sum of Digits]

Task Statement:

- [Take input from user and give the sum of two digits.]

Explanation:

- [This script will take input from user and will give the following output.]

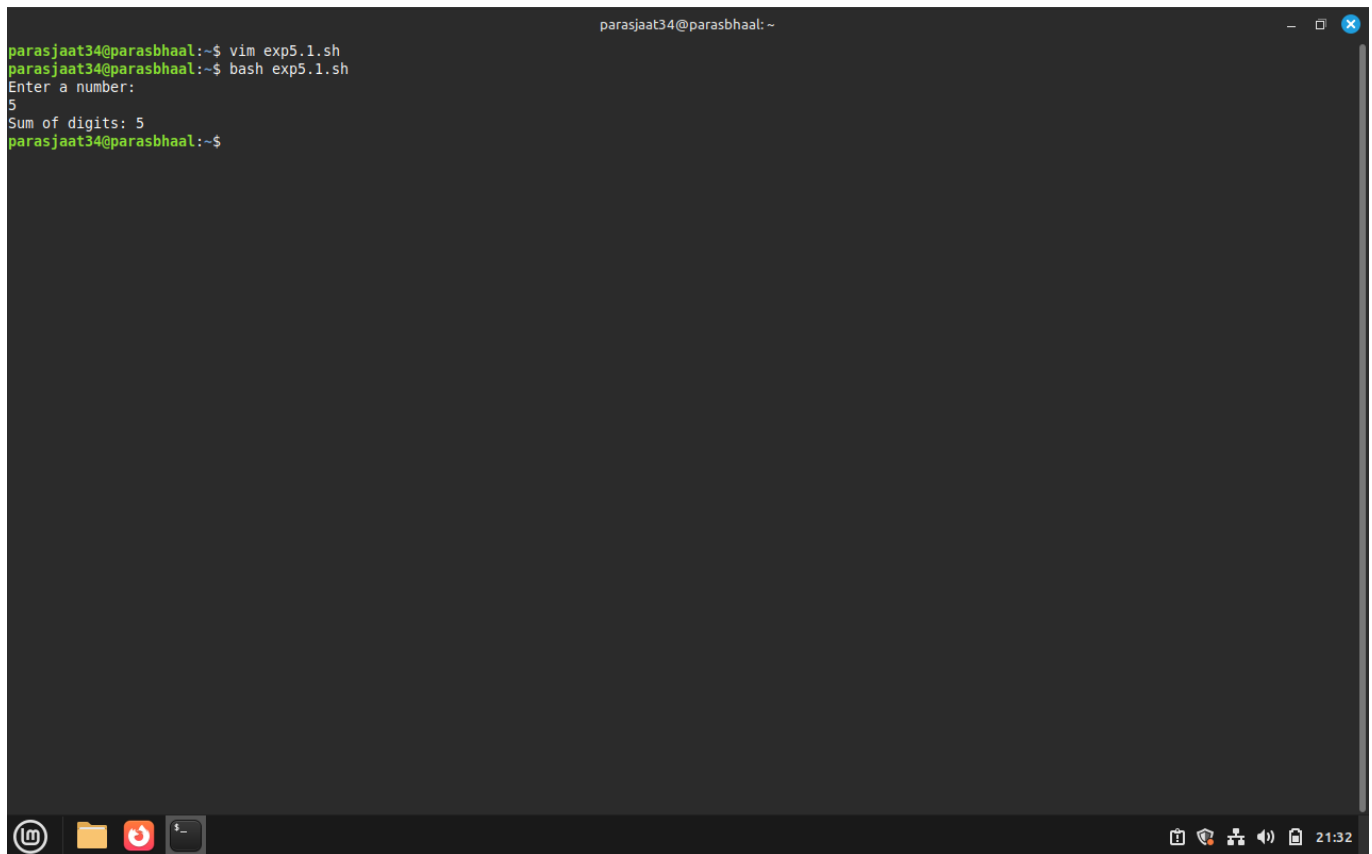
Command(s):

```
#!/bin/bash
echo "Enter a number: "
read num
sum=0

while [ $num -gt 0 ]
do
    digit=$((num % 10))
    sum=$((sum + digit))
    num=$((num / 10))
done

echo "Sum of digits: $sum"
```

Output:



```
parasjaat34@parasbhaal:~$ vim exp5.1.sh
parasjaat34@parasbhaal:~$ bash exp5.1.sh
Enter a number:
5
Sum of digits: 5
parasjaat34@parasbhaal:~$
```

Exercise 3: [Armstrong Numbers]

Task Statement:

- [Take input user and give the sum of Armstrong number of n digits is a number equal to the sum of its digits raised to the power n. Example: $153 = 1^3 + 5^3 + 3^3$]

Explanation:

- [This script will tell if the number entered by the user is an armstrong number or not.]

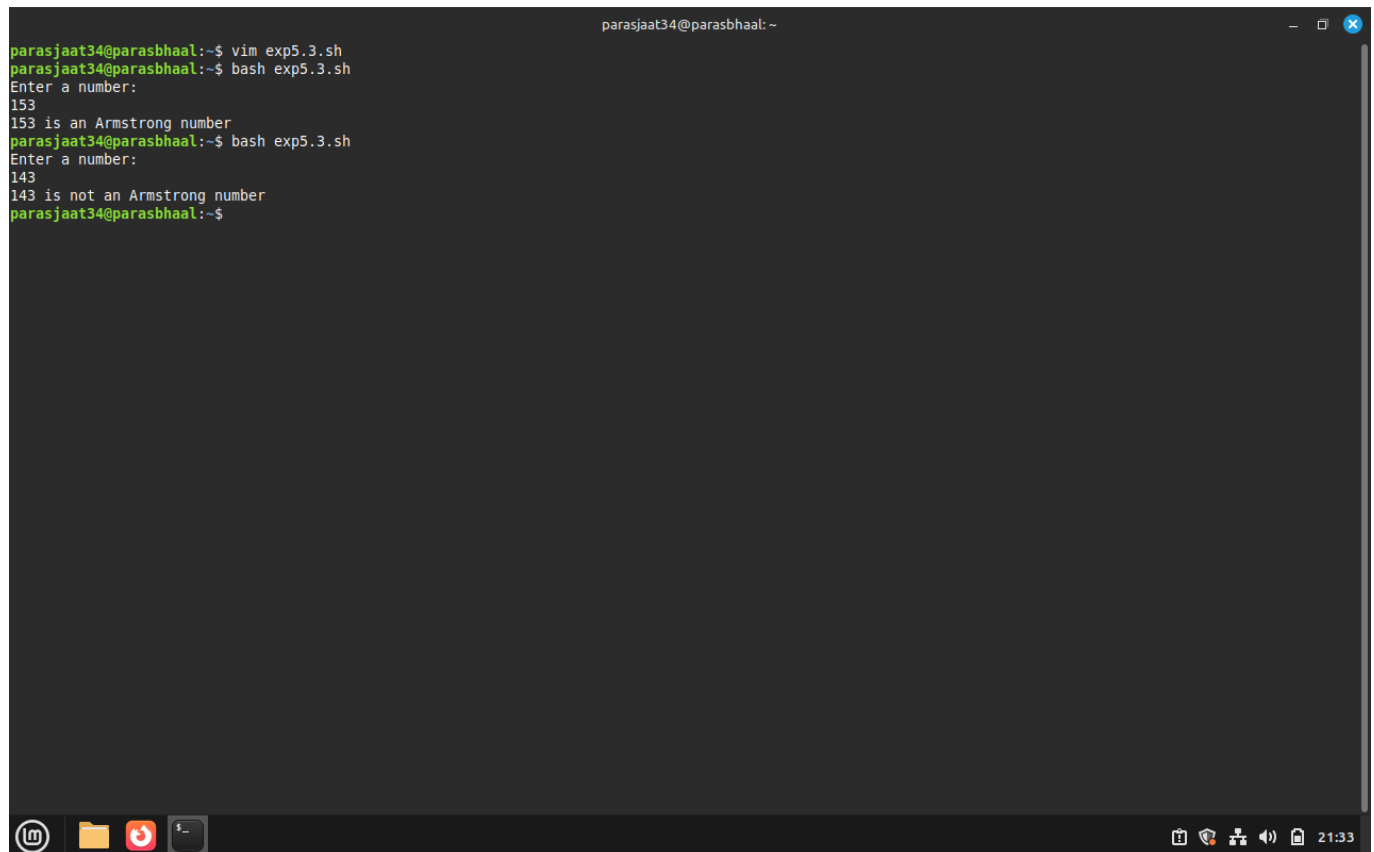
Command(s):

```
#!/bin/bash
echo "Enter a number: "
read num
temp=$num
n=${#num} # number of digits
sum=0

while [ $temp -gt 0 ]
do
    digit=$((temp % 10))
    sum=$((sum + digit**n))
    temp=$((temp / 10))
done
```

```
if [ $sum -eq $num ]
then
    echo "$num is an Armstrong number."
else
    echo "$num is not an Armstrong number."
fi
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' showing the execution of a script. The user runs 'vim exp5.3.sh' and then 'bash exp5.3.sh'. The script prompts 'Enter a number:' and the user enters '153'. The script outputs '153 is an Armstrong number'. The user then runs the script again, enters '143', and the script outputs '143 is not an Armstrong number'. The terminal window has a dark background with green and white text. The bottom of the window shows a taskbar with icons for a terminal, files, a red circle, and a terminal icon, along with system icons on the right.

```
parasjaat34@parasbhaal:~$ vim exp5.3.sh
parasjaat34@parasbhaal:~$ bash exp5.3.sh
Enter a number:
153
153 is an Armstrong number
parasjaat34@parasbhaal:~$ bash exp5.3.sh
Enter a number:
143
143 is not an Armstrong number
parasjaat34@parasbhaal:~$
```

Result:

- The Exercises were successfully completed for Basic Shell Scripting.

Experiment 6: Shell Loops

Name:Paras bhall Roll No.: 590029319 Date: 2025-10-30

Aim:

- To understand and implement shell loops (**for**, **while**, **until**) in Bash.
- To practice loop control constructs (**break**, **continue**) and loop-based file processing.

Requirements

- A Linux system with bash shell.
- A text editor (nano, vim) and permission to create and execute shell scripts.

Theory

Loops allow repeated execution of commands until a condition is met. Common loop constructs in Bash include **for** (iterate over items), **while** (repeat while condition true), and **until** (repeat until condition becomes true). Loop control statements like **break** and **continue** change the flow inside loops. Loops are essential for automating repetitive tasks such as processing multiple files, generating sequences, and collecting user input.

Procedure & Observations

Exercise 1: Simple **for** loop

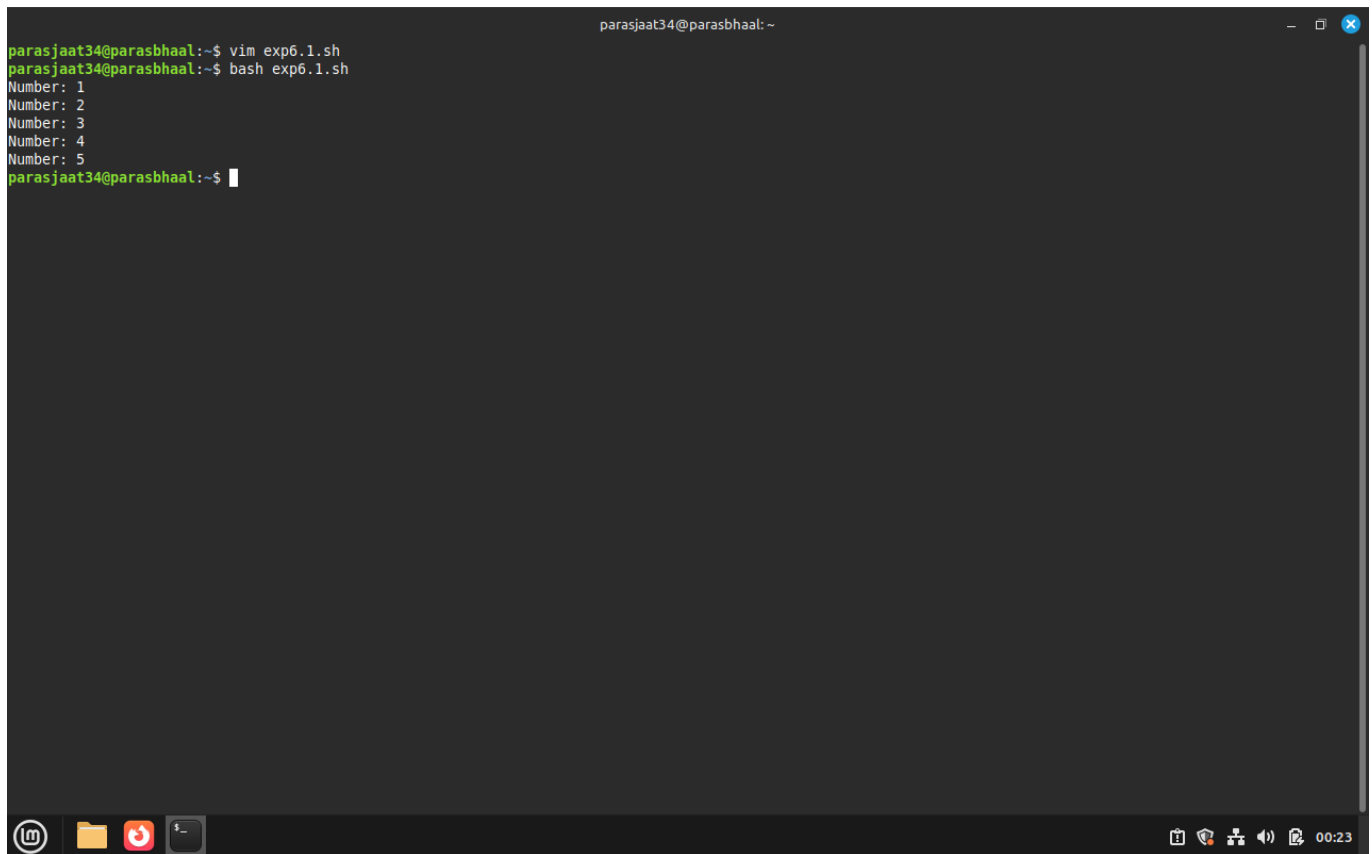
Task Statement:

Write a **for** loop that prints numbers 1 to 5.

Command(s):

```
for i in 1 2 3 4 5; do
    echo "Number: $i"
done
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' with standard window controls. The terminal shows a user running 'vim exp6.1.sh' followed by 'bash exp6.1.sh'. The script outputs five lines: 'Number: 1', 'Number: 2', 'Number: 3', 'Number: 4', and 'Number: 5'. The prompt returns to 'parasjaat34@parasbhaal:~\$'. The bottom of the window shows a taskbar with icons for a terminal, folder, YouTube, and a terminal icon, along with system status icons and a timer at 00:23.

```
parasjaat34@parasbhaal:~$ vim exp6.1.sh
parasjaat34@parasbhaal:~$ bash exp6.1.sh
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
parasjaat34@parasbhaal:~$
```

Exercise 2: **for** loop over files

Task Statement:

Process all **.txt** files in a directory and count lines in each.

Command(s):

```
for f in *.txt; do
    echo "File: $f - Lines: $(wc -l < "$f")"
done
```

Output:

```
parasjaat34@parasbhaal:~$ vim exp6.2.sh
parasjaat34@parasbhaal:~$ bash exp6.2.sh
File: abc.txt - Lines: 0
File: backup_file.txt - Lines: 0
File: config.txt - Lines: 1
File: dated_file.txt - Lines: 0
File: file1.txt - Lines: 0
File: file2.txt - Lines: 0
File: file3.txt - Lines: 0
File: file.txt - Lines: 0
File: logfile.txt - Lines: 0
File: me.txt - Lines: 4
File: newfile.txt - Lines: 0
File: number1.txt - Lines: 20
wc: 'standard input': Is a directory
File: readme.txt - Lines: 0
File: script2.txt - Lines: 4
File: summary.txt - Lines: 4
File: system_info.txt - Lines: 13
File: system.txt - Lines: 1
wc: 'standard input': Is a directory
File: todo.txt - Lines: 0
wc: 'standard input': Is a directory
File: touch.txt - Lines: 0
parasjaat34@parasbhaal:~$
```

Exercise 3: C-style **for** loop

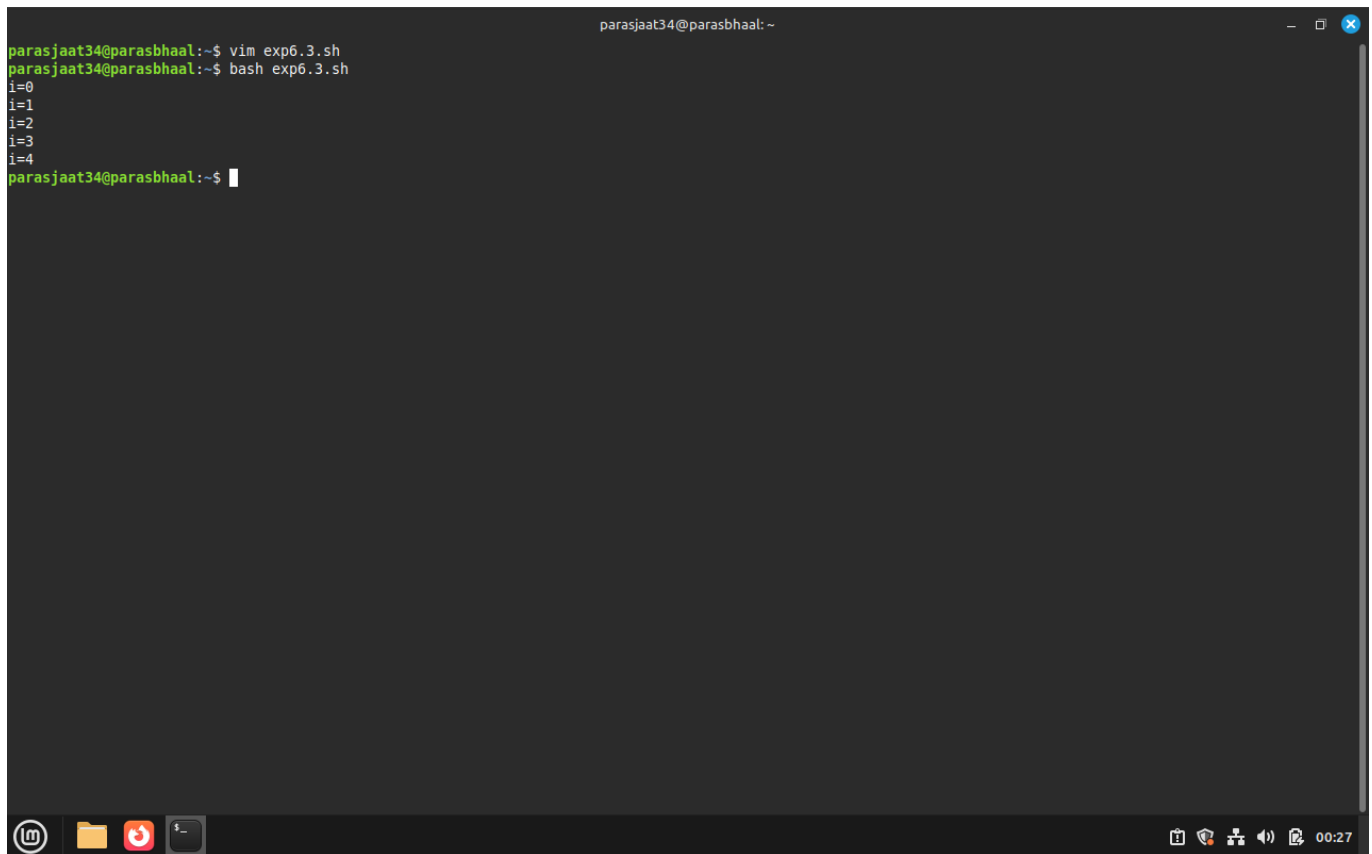
Task Statement:

Use arithmetic C-style loop for numeric iteration.

Command(s):

```
for ((i=0;i<5;i++)); do
    echo "i=$i"
done
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' showing the execution of a script. The user runs 'vim exp6.3.sh' and then 'bash exp6.3.sh'. The script contains a while loop with 'i=0' and 'i=4' as boundaries, incrementing 'i' by 1 in each iteration. The output shows the values of 'i' from 0 to 4. The terminal has a dark background with green and white text. The bottom of the window shows a taskbar with icons for a terminal, folder, and other applications, along with system icons and a timer showing '00:27'.

```
parasjaat34@parasbhaal:~$ vim exp6.3.sh
parasjaat34@parasbhaal:~$ bash exp6.3.sh
i=0
i=1
i=2
i=3
i=4
parasjaat34@parasbhaal:~$
```

Exercise 4: **while** loop and reading input

Task Statement:

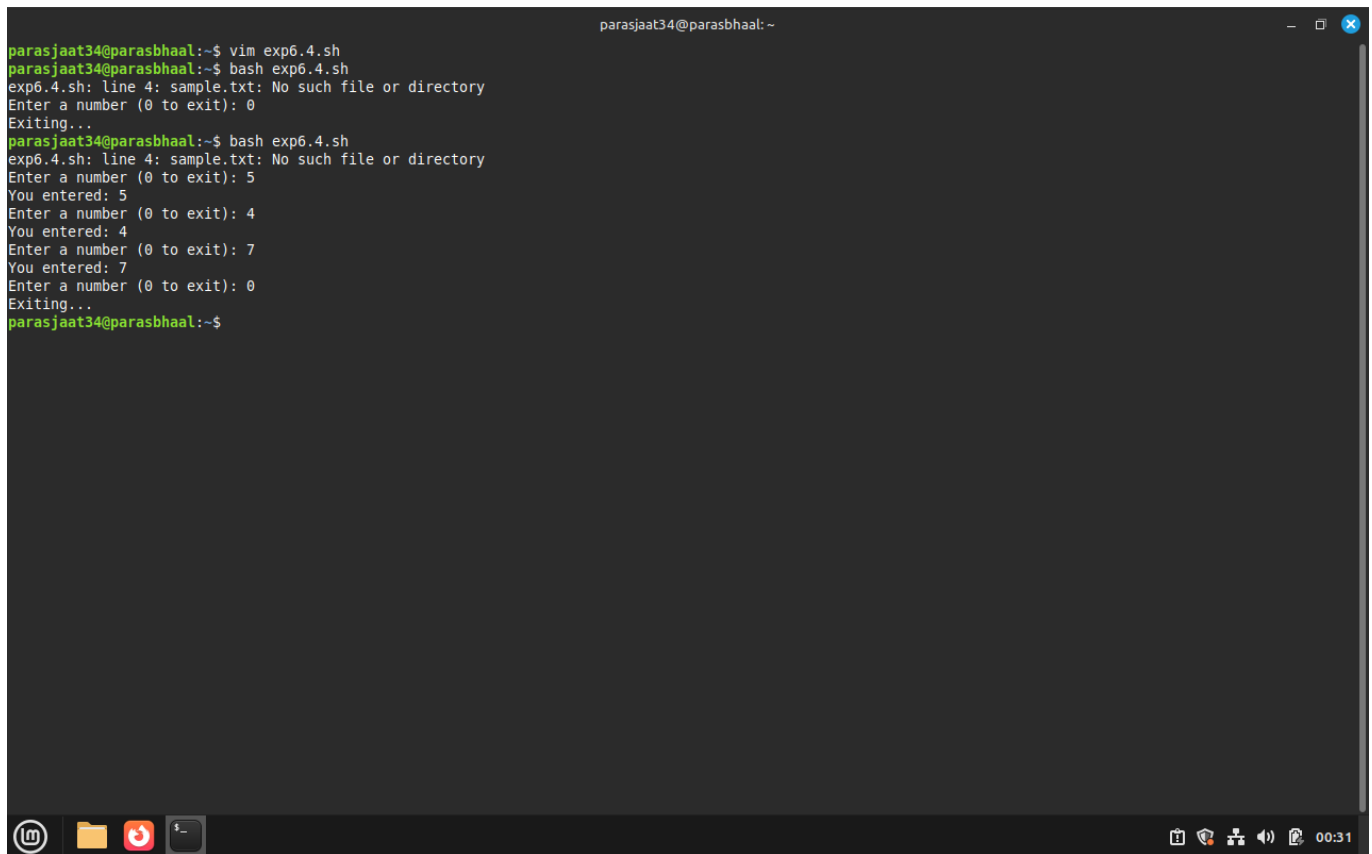
Write a **while** loop that reads lines from a file or from user input.

Command(s):

```
# Read from file
while read -r line; do
    echo "Line: $line"
done < sample.txt

# Read from user with exit condition
while true; do
    read -p "Enter a number (0 to exit): " n
    if [[ $n -eq 0 ]]; then
        echo "Exiting..."; break
    fi
    echo "You entered: $n"
done
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' showing the execution of a script 'exp6.4.sh'. The user first runs 'vim exp6.4.sh' and then 'bash exp6.4.sh'. The script outputs an error: 'exp6.4.sh: line 4: sample.txt: No such file or directory'. It then prompts 'Enter a number (0 to exit): 0' and 'Exiting...'. The user runs the script again, and it prompts for a number. The user enters 5, 4, 7, and 0 in sequence, each followed by 'You entered:' and 'Exiting...'. The terminal window has a dark background and a standard Linux desktop taskbar at the bottom with icons for a terminal, folder, and other applications.

```
parasjaat34@parasbhaal:~$ vim exp6.4.sh
parasjaat34@parasbhaal:~$ bash exp6.4.sh
exp6.4.sh: line 4: sample.txt: No such file or directory
Enter a number (0 to exit): 0
Exiting...
parasjaat34@parasbhaal:~$ bash exp6.4.sh
exp6.4.sh: line 4: sample.txt: No such file or directory
Enter a number (0 to exit): 5
You entered: 5
Enter a number (0 to exit): 4
You entered: 4
Enter a number (0 to exit): 7
You entered: 7
Enter a number (0 to exit): 0
Exiting...
parasjaat34@parasbhaal:~$
```

Exercise 5: `until` loop

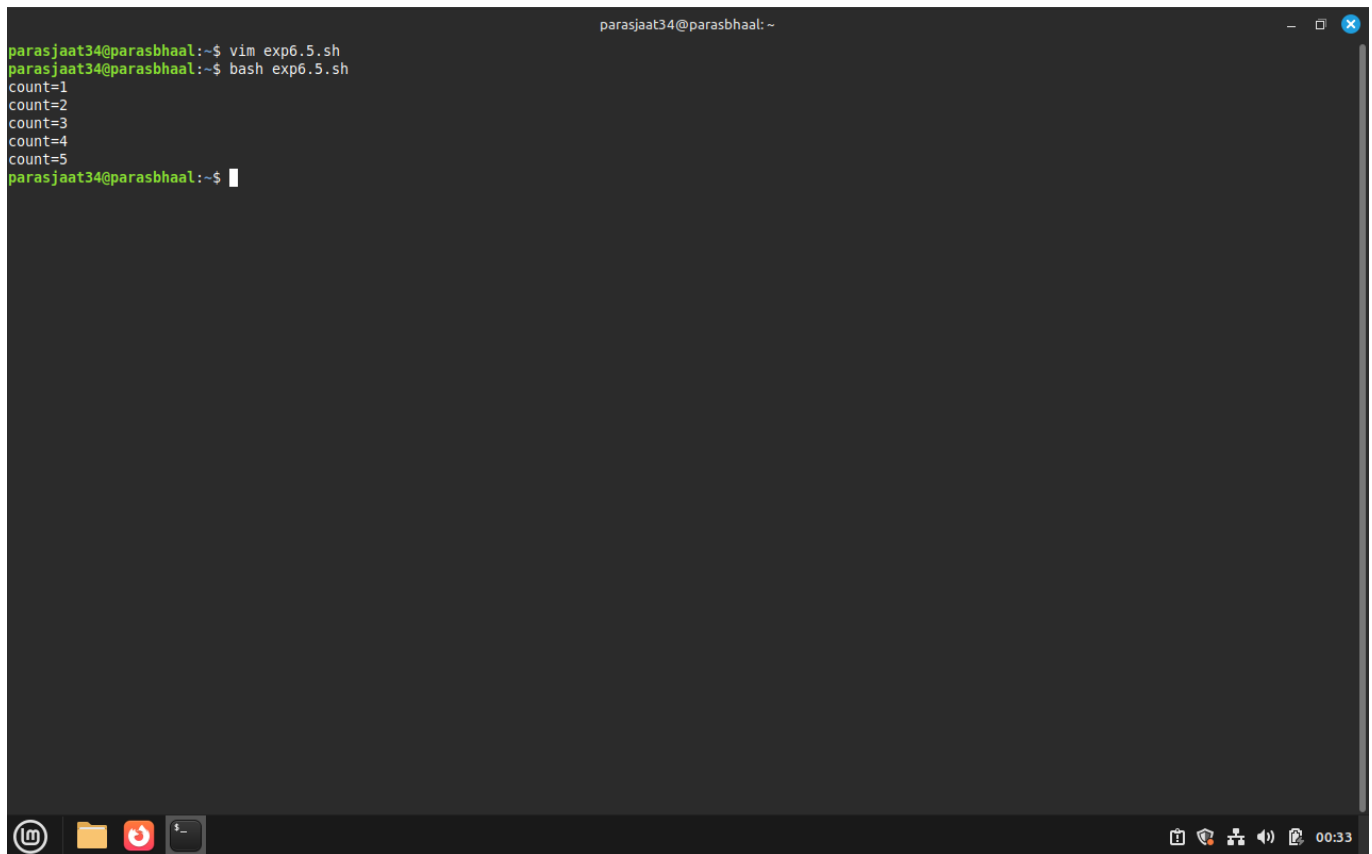
Task Statement:

Use an `until` loop to run until a condition becomes true.

Command(s):

```
count=1
until [ $count -gt 5 ]; do
    echo "count=$count"
    ((count++))
done
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' showing the execution of a script. The user runs 'vim exp6.5.sh' and then 'bash exp6.5.sh'. The script outputs a counter from 1 to 5. The terminal has a dark background with green and orange text. The bottom status bar shows system icons and the time '00:33'.

```
parasjaat34@parasbhaal:~$ vim exp6.5.sh
parasjaat34@parasbhaal:~$ bash exp6.5.sh
count=1
count=2
count=3
count=4
count=5
parasjaat34@parasbhaal:~$
```

Exercise 6: **break** and **continue**

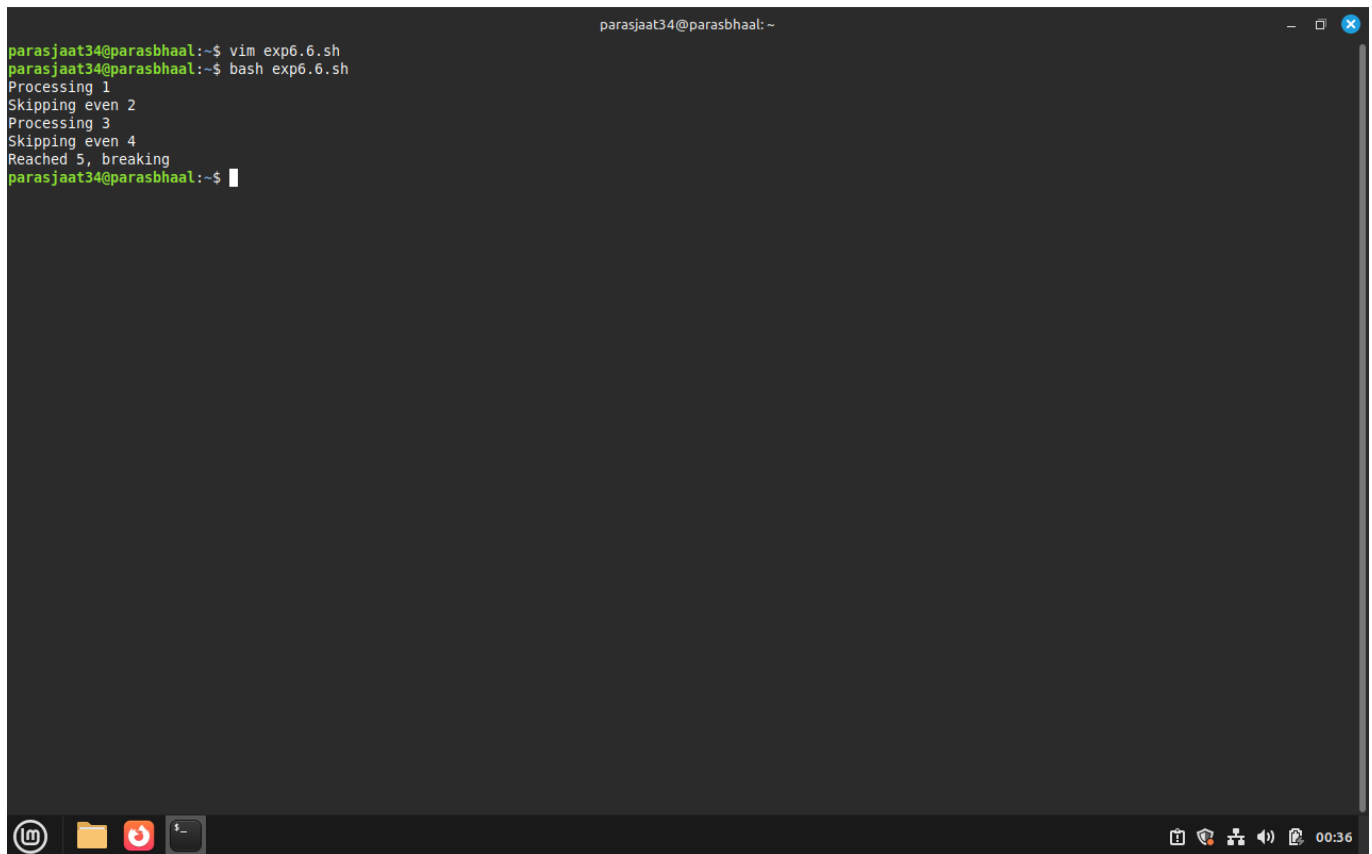
Task Statement:

Demonstrate **break** and **continue** inside a loop.

Command(s):

```
for i in {1..10}; do
    if [[ $i -eq 5 ]]; then
        echo "Reached 5, breaking"; break
    fi
    if (( i % 2 == 0 )); then
        echo "Skipping even $i"; continue
    fi
    echo "Processing $i"
done
```

Output:

A terminal window with a dark background. The prompt is 'parasjaat34@parasbhaal: ~'. The user enters 'vim exp6.6.sh' and then 'bash exp6.6.sh'. The script outputs: 'Processing 1', 'Skipping even 2', 'Processing 3', 'Skipping even 4', 'Reached 5, breaking'. The prompt returns to '~\$'. The terminal has standard window controls at the top right and a taskbar at the bottom with icons for a terminal, folder, and other applications. A system tray at the bottom right shows icons for network, volume, and a timer at 00:36.

```
parasjaat34@parasbhaal:~$ vim exp6.6.sh
parasjaat34@parasbhaal:~$ bash exp6.6.sh
Processing 1
Skipping even 2
Processing 3
Skipping even 4
Reached 5, breaking
parasjaat34@parasbhaal:~$
```

Exercise 7: Nested loops

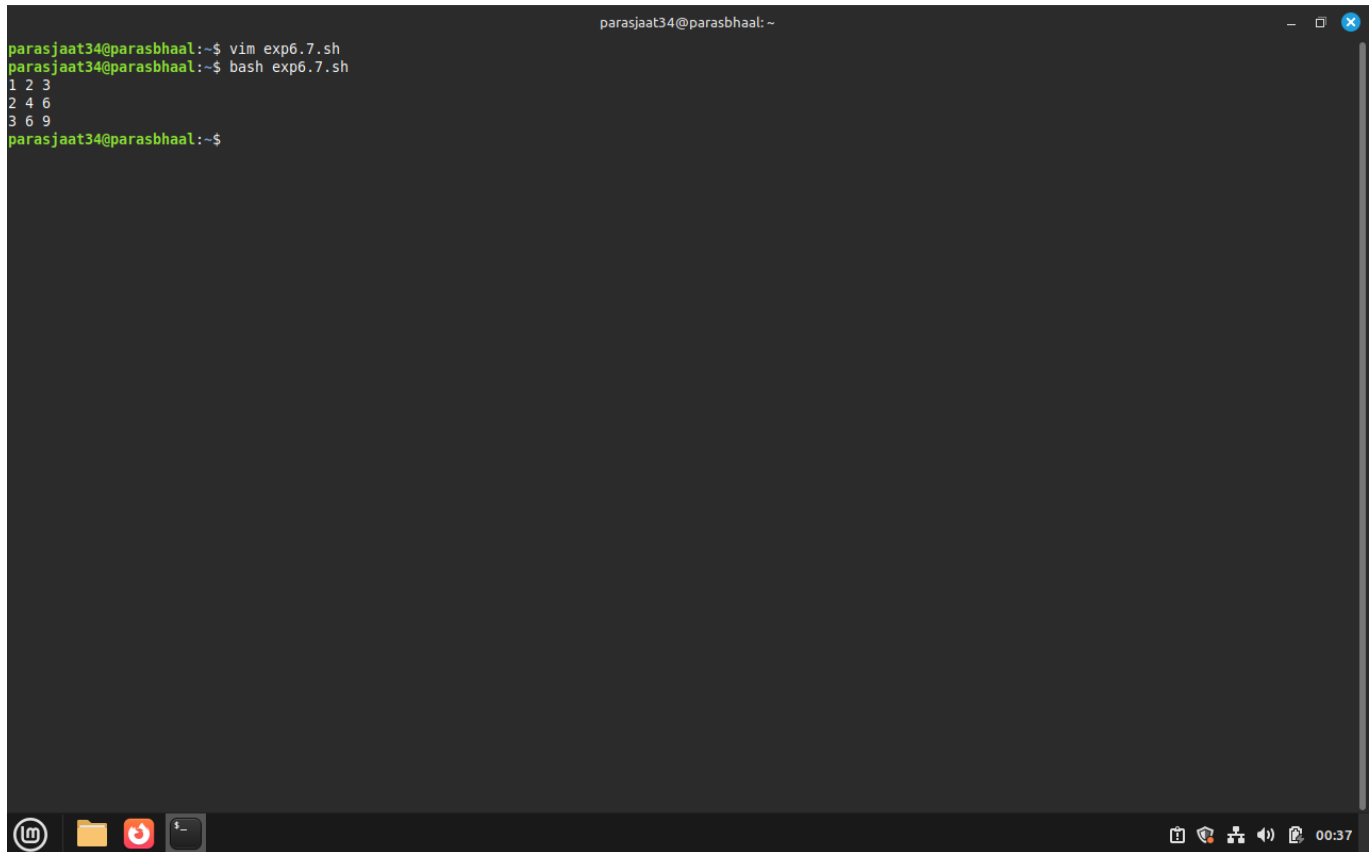
Task Statement:

Create nested loops to generate a multiplication table.

Command(s):

```
for i in {1..3}; do
  for j in {1..3}; do
    echo -n "${(i*j)} "
  done
done
```

Output:

A terminal window with a dark background. The prompt is 'parasjaat34@parasbhaal: ~'. The user enters 'vim exp6.7.sh' and then 'bash exp6.7.sh'. The script outputs three lines: '1 2 3', '2 4 6', and '3 6 9'. The terminal has a title bar with standard window controls and a taskbar at the bottom with icons for a terminal, folder, and other applications. The system clock in the bottom right corner shows '00:37'.

```
parasjaat34@parasbhaal:~$ vim exp6.7.sh
parasjaat34@parasbhaal:~$ bash exp6.7.sh
1 2 3
2 4 6
3 6 9
parasjaat34@parasbhaal:~$
```

Result

- Implemented `for`, `while`, and `until` loops and used loop control statements.
- Practiced reading input, processing files, and nested iteration.

Challenges Faced & Learning Outcomes

- Challenge 1: Handling spaces and special characters when iterating filenames — learned to use quotes and `read -r`.
- Challenge 2: Remembering arithmetic syntax in Bash — used `(( ))` and `expr` where needed.

Learning:

- Loops are powerful for automation in shell scripting. Correct quoting and use of control constructs prevent common bugs.

Conclusion

The lab demonstrated practical loop constructs in Bash for automating repetitive tasks and processing data efficiently.

Experiment 7: Shell Programming, Process and Scheduling

Name: Paras bhall Roll No.: 590029319 Date: 202510-30

Aim:

- To write shell scripts that demonstrate process management.
- To understand how to schedule processes using `cron` and `at`.
- To monitor running processes and practice job control commands.

Requirements

- A Linux machine with bash shell.
- Access to process management commands (`ps`, `top`, `kill`, `jobs`, `fg`, `bg`).
- Access to scheduling utilities (`cron`, `at`).

Theory

Every program running in Linux is a process identified by a unique process ID (PID). Shell programming allows automation of tasks including spawning and controlling processes. Process management commands like `ps`, `top`, `kill`, `jobs`, `bg`, and `fg` let users monitor and control execution. Scheduling utilities such as `cron` (repeated tasks) and `at` (one-time tasks) allow tasks to run automatically at defined times. Combining scripting with scheduling is a core system administration skill.

Procedure & Observations

Exercise 1: Writing a basic shell script

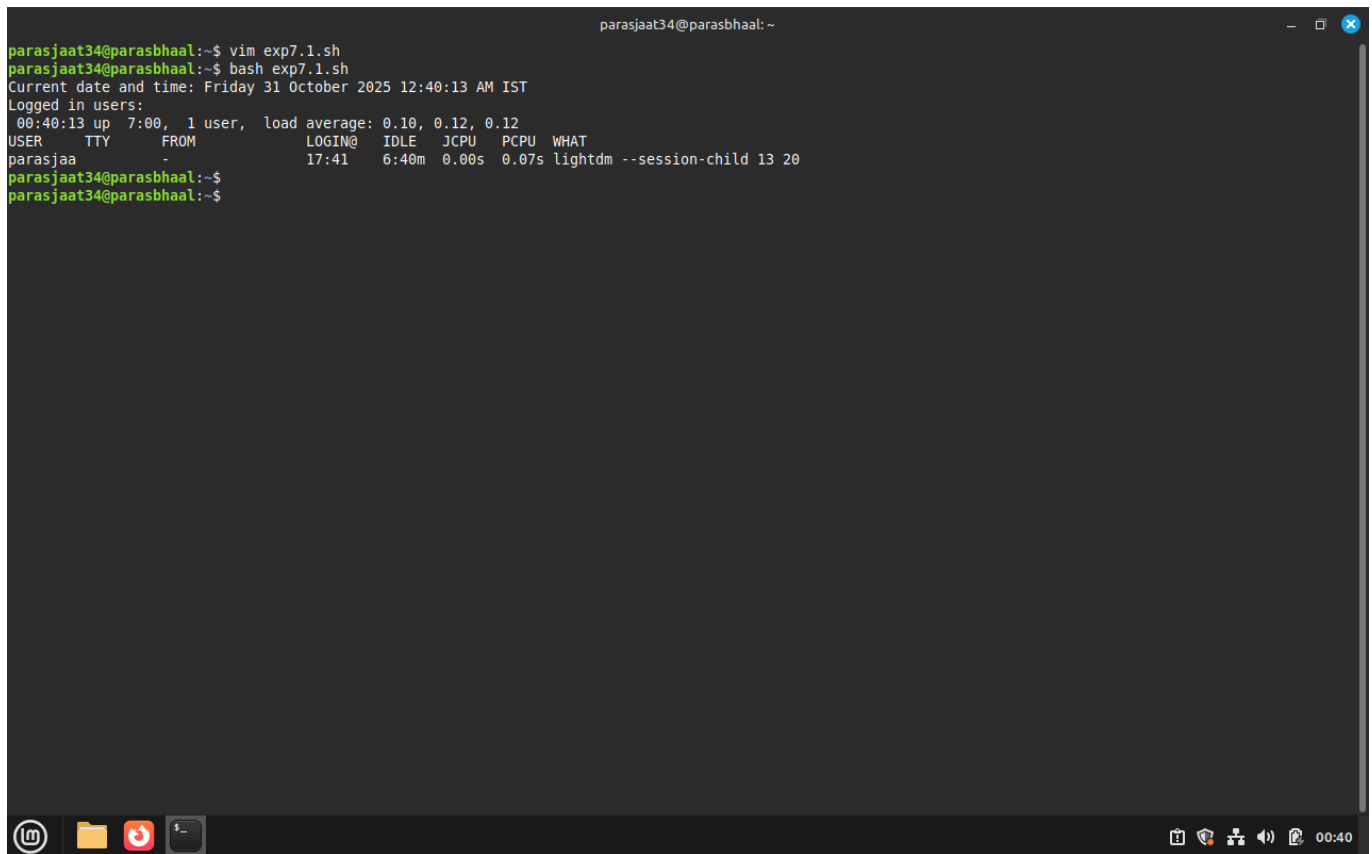
Task Statement:

Create a shell script that prints the current date, time, and the list of logged-in users.

Command(s):

```
#!/bin/bash
echo "Current date and time: $(date)"
echo "Logged in users:"
w
```

Output:

A terminal window titled 'parasjaat34@parasbhaal: ~' showing system boot logs. The output includes the command 'vim exp7.1.sh', 'bash exp7.1.sh', the current date and time 'Friday 31 October 2025 12:40:13 AM IST', and system status 'Logged in users: 00:40:13 up 7:00, 1 user, load average: 0.10, 0.12, 0.12'. A table of system statistics follows, showing user 'parasjaa' logged in at 17:41 with 6:40m idle time and 0.00s CPU time. The terminal window has a dark background and a taskbar at the bottom with icons for a terminal, folder, and other applications.

```
parasjaat34@parasbhaal:~$ vim exp7.1.sh
parasjaat34@parasbhaal:~$ bash exp7.1.sh
Current date and time: Friday 31 October 2025 12:40:13 AM IST
Logged in users:
 00:40:13 up 7:00, 1 user, load average: 0.10, 0.12, 0.12
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
parasjaa  -        -             17:41    6:40m  0.00s  0.07s  lightdm --session-child 13 20
parasjaat34@parasbhaal:~$
parasjaat34@parasbhaal:~$
```

Exercise 2: Background and foreground processes

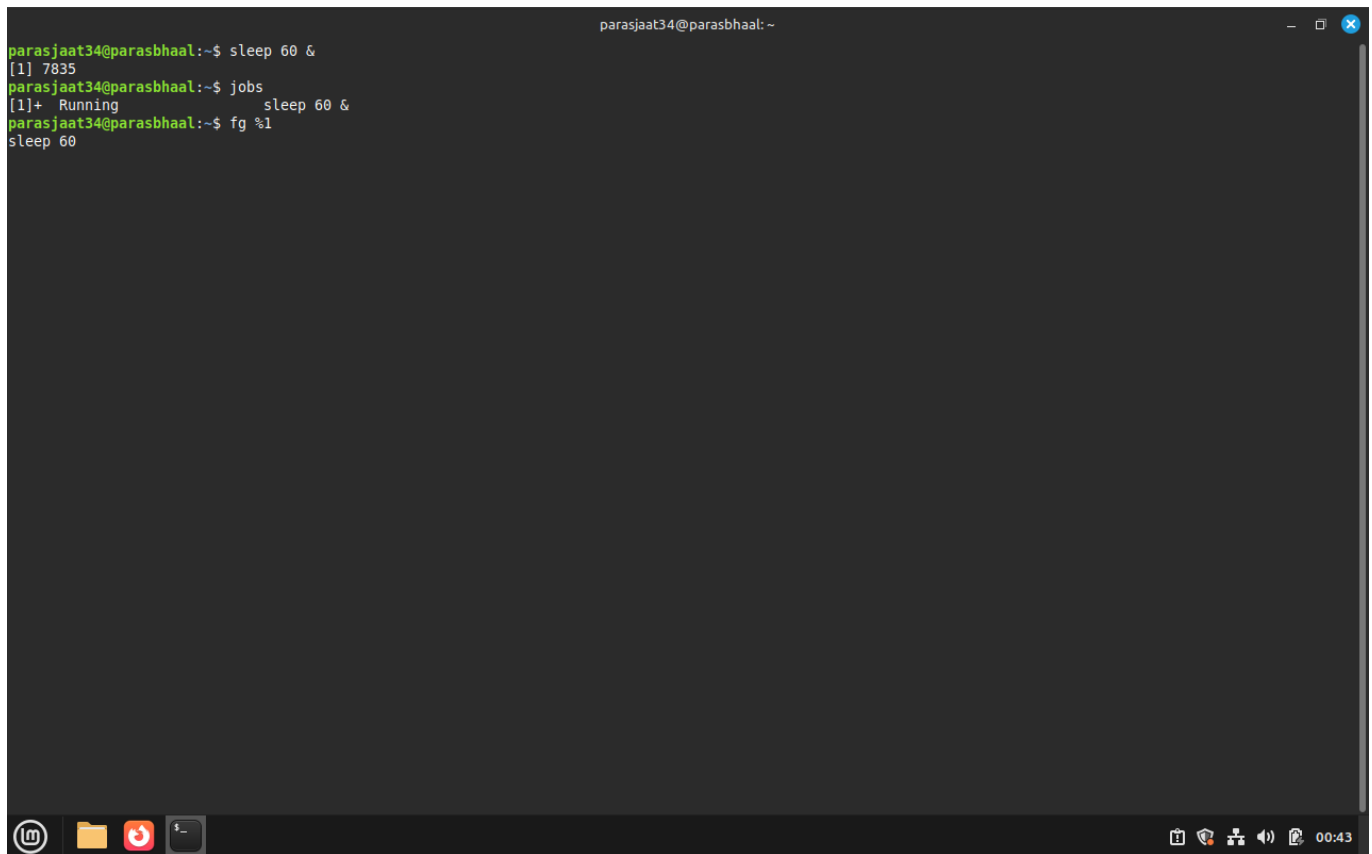
Task Statement:

Run a process in background and bring it to the foreground.

Command(s):

```
sleep 60 &
jobs
fg %1
```

Output:



```
parasjaat34@parasbhaal:~$ sleep 60 &
[1] 7835
parasjaat34@parasbhaal:~$ jobs
[1]+  Running                  sleep 60 &
parasjaat34@parasbhaal:~$ fg %1
sleep 60
```

Exercise 3: Killing a process

Task Statement:

Start a process and terminate it using `kill`.

Command(s):

```
sleep 300 &
ps aux | grep sleep
kill <pid>
```

Output:

```
parasjaat34@parasbhaal:~$ sleep 300 &
[1] 7852
parasjaat34@parasbhaal:~$ ps aux | grep sleep
parasja+  7852  0.0  0.0  8288 1920 pts/0    S   00:44   0:00 sleep 300
parasja+  7855  0.0  0.0  9144 2176 pts/0    S+  00:44   0:00 grep --color=auto sleep
parasjaat34@parasbhaal:~$ kill <pid>
bash: syntax error near unexpected token `newline'
parasjaat34@parasbhaal:~$
```

Exercise 4: Monitoring processes

Task Statement:

Use **ps** and **top** to monitor processes.

Command(s):

```
ps aux | head -5
top
```

Output:

```
parasjaat34@parasbhaal:~$ ps aux | head -5
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.3 22392 13444 ?        Ss   Oct30   0:02 /sbin/init splash
root         2  0.0  0.0      0     0 ?        S    Oct30   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    Oct30   0:00 [pool_workqueue_release]
root         4  0.0  0.0      0     0 ?        I<   Oct30   0:00 [kworker/R-rcu_g]
parasjaat34@parasbhaal:~$ top

top - 00:46:29 up 7:06, 1 user, load average: 0.02, 0.07, 0.08
```

Exercise 5: Using **cron** for scheduling

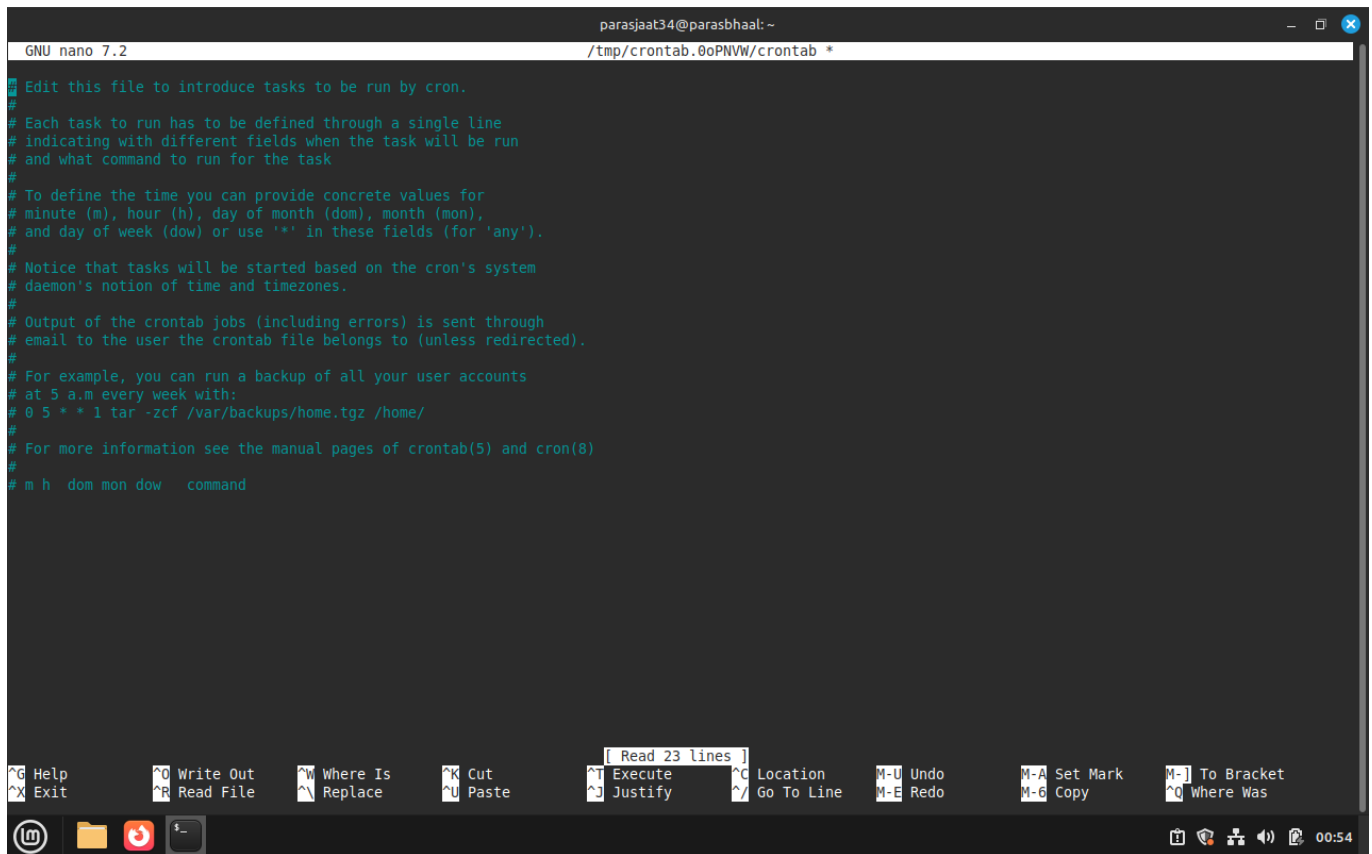
Task Statement:

Schedule a script to run every day at 7:00 AM using **cron**.

Command(s):

```
crontab -e
# Add the following line
0 7 * * * /home/user/myscript.sh
```

Output:



```
GNU nano 7.2 /tmp/crontab.0oPNVW/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
```

Exercise 6: Using **at** for one-time scheduling

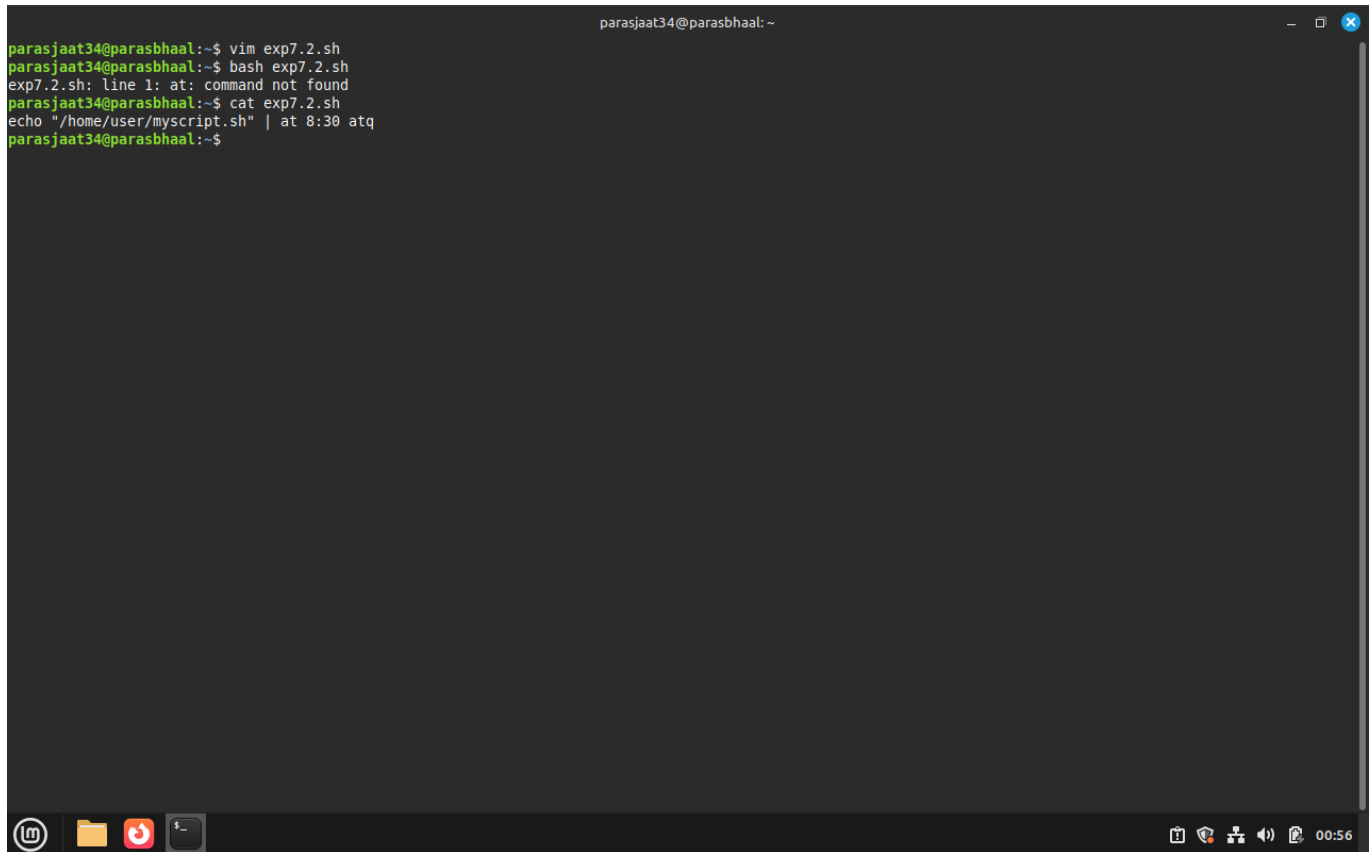
Task Statement:

Schedule a script to run once at a specified time using **at**.

Command(s):

```
echo "/home/user/myscript.sh" | at 08:30
atq
```

Output:



```
parasjaat34@parasbhaal:~$ vim exp7.2.sh
parasjaat34@parasbhaal:~$ bash exp7.2.sh
exp7.2.sh: line 1: at: command not found
parasjaat34@parasbhaal:~$ cat exp7.2.sh
echo "/home/user/myscript.sh" | at 8:30 atq
parasjaat34@parasbhaal:~$
```

The screenshot shows a terminal window with a dark background. The user 'parasjaat34' is at host 'parasbhaal'. They create a file 'exp7.2.sh' using 'vim', then run it with 'bash'. The script contains 'echo "/home/user/myscript.sh" | at 8:30 atq', but an error 'at: command not found' is shown. The terminal window has a title bar and a taskbar at the bottom with various icons and a system clock showing 00:56.

Result

- Learned to create and run shell scripts.
- Managed processes using background, foreground, and kill commands.
- Monitored processes with `ps` and `top`.
- Scheduled recurring tasks with `cron` and one-time tasks with `at`.

Challenges Faced & Learning Outcomes

- Challenge 1: Remembering the `crontab` time format. Solved by using online crontab generators and practice.
- Challenge 2: Ensuring `atd` service is running for `at` command. Fixed by starting the service with `systemctl start atd`.

Learning:

- Gained hands-on knowledge of process creation and termination.
- Learned job control and scheduling using `cron` and `at`.

Conclusion

This experiment provided practical experience with shell scripting, process management, and scheduling. These are critical skills for system administrators to automate and control Linux environments effectively.

Experiment 8: Shell Programming (Continued)

Name: Paras bhall Roll No.: 590029319 Date: 2025-10-30

Aim:

- To extend shell programming concepts by using conditional statements, advanced scripting constructs, and command-line arguments.
- To practice writing scripts that perform decision-making and parameter handling.

Requirements

- A Linux system with bash shell.
- Text editor and permission to create/execute shell scripts.

Theory

Conditional execution in shell scripts allows branching logic using `if`, `elif`, `else`, and `case` statements. Scripts can accept command-line arguments using `$1`, `$2`, ... and `$@` for all arguments. Control flow constructs combined with user input and arguments allow dynamic and reusable scripts.

Procedure & Observations

Exercise 1: Using `if-else`

Task Statement:

Write a script to check whether a given number is positive, negative, or zero.

Explanation:

We used an `if-elif-else` construct to compare the number against 0.

Command(s):

```
#!/bin/bash
num=$1
if [ $num -gt 0 ]; then
    echo "$num is positive"
elif [ $num -lt 0 ]; then
    echo "$num is negative"
else
    echo "$num is zero"
fi
```

Output:



Exercise 2: Using `case`

Task Statement:

Write a script that takes a character as input and classifies it as vowel, consonant, digit, or special character.

Explanation:

The `case` statement provides pattern matching for multiple options.

Command(s):

```
#!/bin/bash
ch=$1
case $ch in
  [aeiouAEIOU]) echo "$ch is a vowel" ;;
  [bcdfghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXYZ]) echo "$ch is a consonant" ;;
  [0-9]) echo "$ch is a digit" ;;
  *) echo "$ch is a special character" ;;
esac
```

Output:



Exercise 3: Command-line arguments

Task Statement:

Write a script that accepts filename(s) as arguments and prints the number of lines in each file.

Explanation:

Command-line arguments are accessed using `$@`. Looping through each argument allows file-wise operations.

Command(s):

```
#!/bin/bash
for file in "$@"; do
  if [ -f "$file" ]; then
    echo "$file: $(wc -l < "$file") lines"
  else
    echo "$file not found"
  fi
done
```

Output:



Exercise 4: Nested conditionals

Task Statement:

Write a script to check if a year is a leap year.

Explanation:

A leap year is divisible by 4, but if divisible by 100 it must also be divisible by 400.

Command(s):

```
#!/bin/bash
year=$1
if (( year % 400 == 0 )); then
    echo "$year is a leap year"
elif (( year % 100 == 0 )); then
    echo "$year is not a leap year"
elif (( year % 4 == 0 )); then
    echo "$year is a leap year"
else
    echo "$year is not a leap year"
fi
```

Output:



Result

- Implemented conditional statements (`if-else`, `case`) in shell scripts.
- Practiced handling command-line arguments and nested conditions.
- Wrote reusable and flexible shell scripts.

Challenges Faced & Learning Outcomes

- Challenge 1: Forgetting to quote variables in conditions — resolved by using `"$var"` to avoid word splitting.
- Challenge 2: Pattern matching in `case` — practiced with multiple examples.

Learning:

- Learned practical use of branching and decision-making in shell scripting.
- Understood command-line argument handling for automation.

Conclusion

This experiment extended shell programming by introducing decision-making and parameter handling. The scripts demonstrate the flexibility of shell programming for different use cases.